*

Contents


* 1

**Cours : Maîtriser Claude Code pour Développeurs** 4
Objectif pédagogique global . . . 4

**PARTIE 1 : FONDAMENTAUX** 4
Objectif de la Partie 1 . . . 4
Module 1 : Introduction à Claude Code . . . 4
1.1 Qu'est-ce qu'un assistant de codage IA ? . . . 4
1.2 Qu'est-ce que Claude Code ? . . . 5
1.3 Comment fonctionne Claude Code ? . . . 6
1.4 Les 5 concepts fondamentaux . . . 6
1.5 Prérequis et coûts . . . 7
1.6 Démo : Claude Code en action . . . 8
Module 2 : Installation et Premier Lancement (30 min) . . . 8
2.1 Installation . . . 8
2.2 Authentification . . . 9
2.3 Premier lancement . . . 9
Module 3 : CLI - Flags et Options essentiels (45 min) . . . 10
3.1 Flags de session . . . 10
3.2 Flags de mode . . . 11
3.3 Flags de configuration . . . 12
3.4 Flags avancés (aperçu) . . . 12
Module 4 : Tools - Les outils natifs (45 min) . . . 13
4.1 Comprendre les Tools . . . 13
4.2 Tools de lecture et recherche . . . 13
4.3 Tools de modification . . . 14
4.4 Tools d'orchestration . . . 14
4.5 Permissions et confirmations . . . 15
Module 5 : Slash Commands natives (30 min) . . . 15
5.1 Commandes de session . . . 15
5.2 Commandes de contexte . . . 16
5.3 Commandes de workflow . . . 17
5.4 Commandes de configuration . . . 17
5.5 Commandes avancées . . . 18
Module 6 : Raccourcis clavier (20 min) . . . 19
6.1 Contrôles généraux . . . 19
6.2 Édition de texte . . . 19
6.3 Navigation et modèles . . . 19
6.4 Raccourcis spéciaux . . . 20
Module 7 : Ajouter du contexte à Claude Code (30 min) . . . 20
7.1 Pourquoi le contexte est crucial . . . 20
7.2 Méthodes pour ajouter du contexte . . . 21
7.3 Contexte persistant avec CLAUDE.md . . . 21
7.4 Ajouter des répertoires de travail . . . 22
Module 8 : Workflow quotidien (45 min) . . . 22
8.1 Explorer une codebase . . . 22
8.2 Modifier du code . . . 23
8.3 Intégration Git . . . 23
8.4 Débugger efficacement . . . 24
8.5 Écrire des tests . . . 24

8.6 Documenter . . . 24
Module 9 : Subagents - Introduction (30 min) . . . 25
9.1 Qu'est-ce qu'un Subagent ? . . . 25
9.2 Subagents intégrés . . . 25
9.3 Observer les Subagents . . . 25
Module 10 : Skills et Plugins - Introduction (30 min) . . . 26
10.1 Qu'est-ce qu'une Skill ? . . . 26
10.2 Utiliser une Skill existante . . . 26
10.3 Qu'est-ce qu'un Plugin ? . . . 27
10.4 Bonnes pratiques débutant . . . 27
Module 11 : MCP Servers - Introduction (30 min) . . . 27
11.1 Qu'est-ce qu'un MCP Server ? . . . 27
11.2 MCP Servers pré-installés . . . 28
11.3 Installer un MCP Server simple . . . 28
11.4 Utiliser les outils MCP . . . 28
Module 12 : Gestion des sessions (20 min) . . . 29
12.1 Sessions et persistance . . . 29
12.2 Historique et navigation . . . 29
12.3 Checkpoints et Rewind . . . 29
Projet de fin de partie gratuite : BizCraft Diagnostic (Phase 0) . . . 30
Contexte du projet . . . 30
Ce qu'on construit . . . 30
Stack technique . . . 30
Étapes de construction avec Claude Code . . . 30
Livrables attendus . . . 31

**PARTIE 2 : MAÎTRISE AVANCÉE** **31**
Objectif de la Partie 2 . . . 31
Module 13 : Configuration avancée - Fichiers (45 min) . . . 32
13.1 Hiérarchie des fichiers de configuration . . . 32
13.2 Le fichier settings.json . . . 32
13.3 Fichiers spéciaux . . . 34
13.4 Configuration par environnement . . . 35
Module 14 : CLAUDE.md et Memory System (45 min) . . . 36
14.1 Le fichier CLAUDE.md . . . 36
14.2 Hiérarchie des fichiers mémoire . . . 37
14.3 Imports et modularité . . . 38
14.4 Bonnes pratiques . . . 39
Module 15 : Contrôler le contexte (30 min) . . . 39
15.1 Comprendre la fenêtre de contexte . . . 39
15.2 Stratégies de compaction . . . 40
15.3 Optimiser l'utilisation du contexte . . . 40
15.4 Gérer les grands projets . . . 41
Module 16 : Permissions et Sécurité (45 min) . . . 41
16.1 Modes de permission . . . 41
16.2 Règles de permission granulaires . . . 42
16.3 Sandboxing . . . 43
16.4 Sécurité en équipe . . . 44
Module 17 : MCP Servers - Maîtrise complète (1h) . . . 45
17.1 Architecture MCP . . . 45
17.2 Gestion des serveurs MCP . . . 45
17.3 MCP Servers essentiels . . . 46
17.4 Créer son propre MCP Server . . . 47
Module 18 : Skills et Subagents personnalisés (1h) . . . 48

18.1 Créer une Skill . . . 49
18.2 Skills avancées . . . 50
18.3 Créer un Subagent personnalisé . . . 51
18.4 Partager en équipe . . . 52
Module 19 : Hooks - Automatisation événementielle (1h15) . . . 53
19.1 Comprendre les Hooks . . . 53
19.2 Événements disponibles . . . 54
19.3 Créer des Hooks . . . 54
19.4 Pièges courants avec les Hooks (Gotchas) . . . 56
19.5 Exemples de hooks utiles . . . 57
19.6 Claude Code SDK . . . 59
Module 20 : IDE Integrations (45 min) . . . 60
20.1 Extension VS Code . . . 60
20.2 Plugin JetBrains . . . 60
20.3 Chrome Integration . . . 60
20.4 Bonnes pratiques IDE . . . 60
Module 21 : Mode Headless et Automatisation (45 min) . . . 61
21.1 Print Mode (−p) . . . 61
21.2 Options avancées . . . 61
21.3 Script exemple . . . 61
21.4 SDK . . . 61
Module 22 : CI/CD - GitHub Actions et GitLab (1h) . . . 61
22.1 GitHub Actions . . . 61
22.2 GitLab CI . . . 62
Module 23 : Patterns agentiques avancés (45 min) . . . 62
23.1 Plan Mode . . . 62
23.2 Orchestration subagents . . . 62
23.3 Extended Thinking . . . 62
Module 24 : Optimisation et Productivité (45 min) . . . 62
Gestion contexte . . . 62
Réduction tokens . . . 62
Module 25 : Configuration Terminal et UX (30 min) . . . 63
Module 26 : Déploiement Enterprise (30 min) . . . 63
Managed Settings . . . 63
Providers . . . 63
Réseau . . . 63
Projet intermédiaire : GitBar - Outil desktop rapide (2h) . . . 63
Présentation . . . 63
Stack technique . . . 63
Fonctionnalités cibles . . . 64
Workflow avec Claude Code . . . 64
Structure du projet . . . 65
CLAUDE.md pour GitBar . . . 65
Livrables . . . 65
Points d'apprentissage . . . 66
Projet final : BizCraft SaaS Complet . . . 66
Phase 1 — MVP . . . 66
Phase 2 — Avancé . . . 66
Évaluation . . . 66

**RESSOURCES** **66**

**ANNEXE : RÉFÉRENCE RAPIDE** **67**
CLI Flags . . . 67

Slash Commands . . . . . 67
Raccourcis . . . . . 67

# Cours : Maîtriser Claude Code pour Développeurs

**Formateur** : James K. GAGLO **Durée totale** : ~14h **Public cible** : Développeurs (junior à senior) **Prérequis** : Connaissances de base en programmation, familiarité avec le terminal

---

## Objectif pédagogique global

À l'issue de cette formation, l'apprenant sera capable de :

1. **Utiliser Claude Code au quotidien** pour explorer, modifier et débugger du code de manière autonome
2. **Configurer un environnement Claude Code optimisé** adapté à son projet et à son équipe
3. **Étendre les capacités de Claude Code** avec des skills personnalisées, des hooks d'automatisation et des serveurs MCP
4. **Intégrer Claude Code dans des workflows CI/CD** pour automatiser les tâches de développement
5. **Construire une application SaaS complète** (BizCraft) en utilisant Claude Code comme assistant principal

---

# PARTIE 1 : FONDAMENTAUX

### Objectif de la Partie 1

À l'issue de cette partie, l'apprenant sera capable de : - Installer et configurer Claude Code sur sa machine - Naviguer efficacement dans l'interface CLI avec les raccourcis et commandes - Utiliser les outils natifs (Read, Edit, Bash, Grep, Glob) pour des tâches quotidiennes - Explorer, modifier et débugger du code avec Claude Code - Créer des commits Git structurés avec Claude Code - Configurer le contexte projet avec CLAUDE.md - Comprendre le rôle des subagents, skills et MCP servers

---

## Module 1 : Introduction à Claude Code

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'expliquer ce qu'est Claude Code, de le positionner par rapport aux autres outils IA de développement, et de comprendre ses concepts fondamentaux (Tools, Slash Commands, Skills, Subagents, MCP Servers).

### 1.1 Qu'est-ce qu'un assistant de codage IA ?

L'évolution des outils d'aide au développement a connu plusieurs phases majeures :

**Phase 1 : Autocomplétion basique** - Suggestions basées sur des patterns syntaxiques - Complétion de noms de variables et fonctions - Aucune compréhension sémantique du code

**Phase 2 : Copilots (2021-2023)** - GitHub Copilot, TabNine, Codeium - Suggestions de lignes ou blocs de code - Compréhension limitée du contexte projet - Mode "passif" : suggère, l'humain décide et exécute

**Phase 3 : Assistants conversationnels (2023-2024)** - ChatGPT, Claude.ai, Gemini - Dialogue en langage naturel sur le code - Génération de snippets à copier-coller - Pas d'accès direct au système de fichiers

**Phase 4 : Agents autonomes (2024+)** - Claude Code, Cursor Composer, Aider - Accès complet au projet et au terminal - Exécution autonome de tâches complexes - Mode “actif” : l’agent explore, modifie, teste

**Les 3 catégories d’outils IA pour le code :**

| Catégorie | Exemples | Mode d’interaction | Autonomie |
|---|---|---|---|
| Copilots | GitHub Copilot, TabNine | Inline suggestions | Faible |
| Assistants | Claude.ai, ChatGPT | Chat + copier-coller | Moyenne |
| Agents | Claude Code, Cursor, Aider | Terminal/IDE complet | Élevée |

### 1.2 Qu’est-ce que Claude Code ?

Claude Code est un **agent de développement autonome** qui s’exécute dans votre terminal. Contrairement à Claude (le chatbot web), Claude Code :

- A accès à votre système de fichiers
- Peut exécuter des commandes shell
- Comprend le contexte de votre projet entier
- Peut modifier, créer et supprimer des fichiers
- Peut lancer des tests, commits, builds

**Différences clés avec Claude (chat web) :**

| Aspect | Claude (chat) | Claude Code |
|---|---|---|
| Accès fichiers | Non | Oui |
| Exécution shell | Non | Oui |
| Contexte projet | Copier-coller | Automatique |
| Modifications | Manuelles | Directes |
| Persistance | Session web | Sessions sauvegardées |

**Positionnement vs concurrents :**

| Outil | Type | Points forts | Points faibles |
|---|---|---|---|
| **Claude Code** | CLI agentique | Autonomie, qualité raisonnement | Terminal uniquement |
| GitHub Copilot | Extension IDE | Intégration IDE native | Suggestions limitées |
| Cursor | IDE complet | Interface visuelle | IDE propriétaire |
| Cody | Extension | Gratuit, multi-IDE | Moins autonome |
| Aider | CLI | Open source | Moins de contexte |

**Cas d’usage idéaux pour Claude Code :**

1. **Exploration de codebase** : “Explique-moi comment fonctionne l’authentification dans ce projet”
2. **Refactoring** : “Refactorise cette fonction pour utiliser des async/await”
3. **Debugging** : “Trouve pourquoi ce test échoue et corrige le bug”
4. **Génération** : “Crée les tests unitaires pour ce service”
5. **Documentation** : “Génère la documentation API pour ces endpoints”

### 1.3 Comment fonctionne Claude Code ?

**Architecture simplifiée :**

```
┌──────────────────────────────────────────────────────────┐
│                      CLAUDE CODE                         │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│   │   Modèle     │     │  Contexte    │     │   Tools      │
│   │   Claude     │◄──►│   Projet     │◄──►│  (Actions)   │
│   │  (Sonnet/    │     │  (fichiers,  │     │  Read,Edit   │
│   │   Opus)      │     │   git, etc)  │     │  Bash, etc   │
│   └──────────────┘     └──────────────┘     └──────────────┘
│                                                          │
├──────────────────────────────────────────────────────────┤
│                     VOTRE TERMINAL                       │
│                                                          │
│   ┌──────────────────────────────────────────────┐       │
│   │  Système de fichiers  │  Shell  │  Git  │  etc  │    │
│   └──────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────┘
```

**Le concept d'agent autonome :**

Un agent, contrairement à un simple chatbot : 1. **Observe** : Lit les fichiers, comprend la structure 2. **Planifie** : Décompose la tâche en étapes 3. **Agit** : Exécute des actions (édition, commandes) 4. **Évalue** : Vérifie le résultat 5. **Itère** : Corrige si nécessaire

**Gestion du contexte :**

Claude Code maintient une "fenêtre de contexte" qui inclut : - Votre conversation actuelle - Les fichiers que vous référencez - Les fichiers qu'il a lu ou modifié - Les résultats des commandes exécutées - Le contenu de `CLAUDE.md` (mémoire projet)

**Les 3 modes d'interaction :**

| Mode | Commande | Usage |
|---|---|---|
| **Interactif** | `claude` | Sessions de travail normales |
| **Headless** | `claude -p "..."` | Scripts, automation, CI/CD |
| **Intégré** | Extension VS Code/JetBrains | Dans votre IDE |

### 1.4 Les 5 concepts fondamentaux

Claude Code repose sur 5 concepts clés qu'il faut maîtriser :

**1. Tools (Outils)** Les Tools sont les **capacités d'action** de Claude Code. Ce sont les "mains" de l'agent.

**Tools de lecture :** - `Read` : Lire le contenu d'un fichier - `Glob` : Trouver des fichiers par pattern - `Grep` : Rechercher du texte dans les fichiers - `WebSearch` / `WebFetch` : Rechercher sur internet

**Tools de modification :** - `Write` : Créer un nouveau fichier - `Edit` : Modifier une partie d'un fichier - `Bash` : Exécuter des commandes terminal

**Tools d'orchestration :** - `Task` : Lancer un sous-agent spécialisé - `AskUserQuestion` : Poser une question à l'utilisateur

**2. Slash Commands** Les Slash Commands sont des **raccourcis** pour des actions fréquentes. Elles commencent par `/`.

```
/help      → Affiche l'aide
/commit    → Crée un commit git
/clear     → Efface la conversation
/model     → Change le modèle
```

**3. Skills** Les Skills sont des **prompts packagés** réutilisables. Elles étendent les capacités de Claude Code avec des workflows personnalisés.

```
/code-review    → Lancer une revue de code
/standup        → Générer un rapport de standup
/deploy         → Déployer l'application
```

**4. Subagents** Les Subagents sont des **agents spécialisés** que Claude Code peut lancer pour des tâches spécifiques :

- **Explore** : Explorer et comprendre une codebase
- **Plan** : Planifier une implémentation complexe
- **Bash** : Exécuter des commandes shell
- **general-purpose** : Tâches variées

**5. MCP Servers** Les MCP (Model Context Protocol) Servers sont des **extensions externes** qui donnent de nouveaux outils à Claude Code :

- Accès aux bases de données (PostgreSQL, SQLite)
- Intégrations (GitHub, Slack, Jira)
- Automatisation navigateur (Chrome DevTools)

### 1.5 Prérequis et coûts

**Compte Anthropic requis :**

Pour utiliser Claude Code, vous avez besoin d'un compte Anthropic avec l'une de ces options :

| Plan | Prix | Caractéristiques |
|---|---|---|
| **Pro** | 20$/mois | Usage limité, idéal pour débuter |
| **Max** | Variable | Usage intensif, limites élevées |
| **API** | Pay-as-you-go | Facturation à l'usage |

**Estimation des coûts API :**

| Usage | Coût estimé/mois |
|---|---|
| Léger (1-2h/jour) | 20-50$ |
| Modéré (4h/jour) | 50-150$ |
| Intensif (8h/jour) | 150-400$ |

**Providers alternatifs :**

Claude Code supporte aussi : - **AWS Bedrock** : Pour les entreprises AWS - **Google Vertex AI** : Pour les environnements GCP - **Azure Foundry** : Pour les environnements Azure

### 1.6 Démo : Claude Code en action

Dans cette section, nous allons voir 3 démonstrations pratiques de Claude Code.

**Démo 1 : Explorer un projet inconnu (5 min)**

```
# Lancez Claude Code dans un projet
cd ~/mon-projet-inconnu
claude

# Demandez une vue d'ensemble
> Décris-moi ce projet. Quelle est sa structure,
> sa stack technique, et comment il fonctionne ?
```

Claude va automatiquement : 1. Lire les fichiers de configuration (package.json, go.mod, etc.) 2. Analyser la structure des dossiers 3. Identifier les patterns et frameworks utilisés 4. Vous présenter un résumé clair

**Démo 2 : Corriger un bug (10 min)**

```
> J'ai une erreur "undefined is not a function"
> quand je clique sur le bouton de login.
> Voici la stack trace : [coller la stack trace]
> Trouve la cause et corrige le bug.
```

Claude va : 1. Analyser la stack trace 2. Lire les fichiers concernés 3. Identifier la cause racine 4. Proposer et appliquer la correction 5. Suggérer de lancer les tests

**Démo 3 : Générer des tests (5 min)**

```
> Génère les tests unitaires pour le fichier
> src/services/auth.service.ts
> Utilise Jest et couvre les cas limites.
```

Claude va : 1. Lire le fichier source 2. Comprendre les fonctions à tester 3. Créer un fichier de tests complet 4. Couvrir les cas nominaux et limites

---

## Module 2 : Installation et Premier Lancement (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'installer Claude Code via npm ou Homebrew, de s'authentifier avec son compte Anthropic ou une API key, et de lancer sa première session interactive.

### 2.1 Installation

**Méthode 1 : npm (recommandé)**

```
# Installation globale via npm
npm install -g @anthropic-ai/claude-code

# Vérification
claude --version
```

**Méthode 2 : Homebrew (macOS)**

```
# Installation via Homebrew
brew install claude-code
```

```
# Vérification
claude --version
```

**Mise à jour :**

```
# Mettre à jour Claude Code
claude update

# Ou via npm
npm update -g @anthropic-ai/claude-code
```

**Configuration requise :** - Node.js 18+ - Terminal compatible (iTerm2, Terminal.app, Windows Terminal, etc.) - Connexion internet

### 2.2 Authentification

**Option 1 : Connexion avec compte Anthropic (recommandé)**

```
# Premier lancement - ouvre un navigateur pour l'auth
claude

# Suivez les instructions à l'écran
```

**Option 2 : API Key**

```
# Définir la clé API dans votre shell
export ANTHROPIC_API_KEY="sk-ant-..."

# Ou dans un fichier .env
echo 'ANTHROPIC_API_KEY=sk-ant-...' >> ~/.zshrc
source ~/.zshrc
```

**Vérifier l'authentification :**

```
# Affiche le status de connexion
claude /status
```

### 2.3 Premier lancement

**Démarrer une session interactive :**

```
# Dans le dossier de votre projet
cd ~/mon-projet
claude
```

**Démarrer avec un prompt initial :**

```
# Poser directement une question
claude "Explique-moi ce projet"
```

**Comprendre l'interface terminal :**

```
┌──────────────────────────────────────────────┐
│ Claude Code                                  │
│ Model: claude-sonnet-4-20250514              │
│ Working directory: ~/mon-projet              │
├──────────────────────────────────────────────┤
│                                              │
│ > _                                          │
│                                              │
└──────────────────────────────────────────────┘
```

**Les indicateurs visuels :**

| Indicateur | Signification |
|---|---|
| ● Spinner | Claude réfléchit |
| `Reading...` | Lecture de fichier |
| `Editing...` | Modification en cours |
| `Running...` | Exécution de commande |
| ✓ Checkmark | Action réussie |
| ✗ Croix | Erreur |

**Exercice pratique :**

1. Installez Claude Code avec la méthode de votre choix
2. Authentifiez-vous
3. Naviguez vers un projet existant
4. Lancez `claude`
5. Posez la question : "Qu'est-ce que ce projet fait ?"
6. Observez comment Claude explore les fichiers

---

## Module 3 : CLI - Flags et Options essentiels (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'utiliser les flags CLI essentiels pour gérer ses sessions (`-c`, `-r`), choisir le modèle (`--model`), configurer le mode de permissions (`--permission-mode`), et exécuter des commandes headless (-p).

### 3.1 Flags de session

Ces flags contrôlent comment vous interagissez avec vos sessions de travail.

**`--continue` / `-c` : Reprendre la dernière conversation**

```
# Reprendre exactement où vous vous êtes arrêté
claude -c

# Cas d'usage : Vous avez fermé le terminal par erreur
```

**`--resume` / `-r` : Reprendre une session spécifique**

```
# Par nom de session
claude -r "refactoring-auth"
```

```
# Par ID de session
claude -r abc123

# Lister les sessions disponibles
claude sessions list
```

**`--model` : Choisir le modèle**

```
# Utiliser Sonnet (par défaut, bon équilibre)
claude --model sonnet

# Utiliser Opus (plus puissant, plus cher)
claude --model opus

# Utiliser Haiku (rapide, économique)
claude --model haiku
```

| Modèle | Forces | Coût relatif |
|---|---|---|
| **Haiku** | Rapide, tâches simples | $ |
| **Sonnet** | Équilibré, usage quotidien | $||**Opus**|Raisonnementcomplexe|$$ |

### 3.2 Flags de mode

**`--print` / `-p` : Mode non-interactif (headless)**

```
# Exécuter et quitter immédiatement
claude -p "Liste tous les TODO dans ce projet"

# Parfait pour les scripts
result=$(claude -p "Génère un résumé de CHANGELOG.md")
echo "$result"
```

**`--permission-mode` : Niveau de permissions**

```
# Mode plan : affiche ce qu'il ferait sans exécuter
claude --permission-mode plan

# Mode acceptEdits : auto-accepte les modifications de fichiers
claude --permission-mode acceptEdits

# Mode default : demande confirmation pour tout
claude --permission-mode default
```

| Mode | Fichiers | Commandes | Usage |
|---|---|---|---|
| `default` | Demande | Demande | Normal |
| `plan` | Bloqué | Bloqué | Planification |
| `acceptEdits` | Auto | Demande | Confiance modérée |
| `bypass` | Auto | Auto | Scripts (risqué) |

**`--output-format` : Format de sortie**

```
# Texte brut (défaut)
claude -p "question" --output-format text
```

```
# JSON structuré
claude -p "question" --output-format json

# JSON en streaming
claude -p "question" --output-format stream-json
```

### 3.3 Flags de configuration

**`--add-dir` : Ajouter des répertoires de travail**

```
# Travailler sur plusieurs dossiers
claude --add-dir ../shared-lib --add-dir ../common-types

# Cas d'usage : monorepos, dépendances locales
```

**`--mcp-config` : Charger des serveurs MCP**

```
# Charger une config MCP spécifique
claude --mcp-config ./my-mcp-servers.json
```

**`--settings` : Charger des paramètres supplémentaires**

```
# Charger des settings personnalisés
claude --settings ./project-settings.json
```

### 3.4 Flags avancés (aperçu)

**`--max-turns` : Limiter les tours agentiques**

```
# Maximum 5 tours d'interaction
claude -p "Corrige ce bug" --max-turns 5

# Utile pour éviter les boucles infinies
```

**`--max-budget-usd` : Budget maximum**

```
# Ne pas dépasser 1$ pour cette session
claude --max-budget-usd 1.0

# Utile pour les environnements de CI/CD
```

**`--system-prompt` : Remplacer le prompt système**

```
# Comportement personnalisé
claude --system-prompt "Tu es un expert en sécurité. Analyse uniquement les aspects sécurité du
```

**Exercice pratique :**

1. Lancez Claude Code avec différents modèles et comparez
2. Utilisez le mode `-p` pour obtenir une information rapide
3. Essayez le mode `plan` pour voir ce que Claude ferait
4. Créez un alias bash avec vos flags préférés

```
# Exemple d'alias
alias cc='claude --model sonnet'
alias cco='claude --model opus'
alias ccp='claude -p'
```

---

## Module 4 : Tools - Les outils natifs (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'identifier les outils natifs de Claude Code (Read, Write, Edit, Bash, Glob, Grep, WebSearch), de comprendre quand et comment Claude les utilise, et de gérer les demandes de permissions.

### 4.1 Comprendre les Tools

Les Tools sont les **capacités d'action** de Claude Code. Quand vous demandez quelque chose, Claude choisit automatiquement les outils appropriés.

**Comment Claude choisit les outils :**

1. Analyse votre demande
2. Identifie les actions nécessaires
3. Sélectionne les outils appropriés
4. Exécute dans l'ordre logique

**Exemple de sélection automatique :**

```
Vous : "Ajoute une fonction de validation d'email dans utils.ts"

Claude pense :
1. Je dois d'abord LIRE utils.ts pour comprendre le contexte → Tool: Read
2. Je dois MODIFIER le fichier pour ajouter la fonction → Tool: Edit
3. Je pourrais VÉRIFIER que le code compile → Tool: Bash (npm run build)
```

### 4.2 Tools de lecture et recherche

**Read : Lire le contenu d'un fichier**

```
Vous : "Montre-moi le contenu de src/config.ts"

Claude utilise Read pour afficher le fichier.
```

**Caractéristiques :** - Lit n'importe quel fichier texte - Peut lire des images (Claude est multimodal) - Peut lire des PDFs - Affiche les numéros de ligne

**Glob : Trouver des fichiers par pattern**

```
Vous : "Trouve tous les fichiers de test"

Claude utilise Glob avec le pattern **/*.test.ts
```

**Patterns courants :**

| Pattern | Trouve |
|---|---|
| `**/*.ts` | Tous les fichiers TypeScript |
| `src/**/*.go` | Fichiers Go dans src/ |
| `**/test_*.py` | Tests Python |
| `*.{js,jsx,ts,tsx}` | JS/TS et leurs variantes JSX |

**Grep : Rechercher du texte dans les fichiers**

```
Vous : "Où est-ce qu'on utilise la fonction validateUser ?"

Claude utilise Grep pour trouver toutes les occurrences.
```

**Syntaxe supportée :** - Expressions régulières complètes - Recherche case-insensitive (`-i`) - Contexte autour des matches (`-C`)

**WebSearch / WebFetch : Rechercher sur internet**

```
Vous : "Comment implémenter OAuth2 avec Go ?"

Claude utilise WebSearch pour trouver des ressources récentes.
```

**Cas d'usage :** - Documentation à jour - Solutions à des erreurs - Meilleures pratiques actuelles

### 4.3 Tools de modification

**Write : Créer un nouveau fichier**

```
Vous : "Crée un fichier README.md pour ce projet"

Claude utilise Write pour créer le fichier.
```

**Comportement :** - Crée le fichier s'il n'existe pas - Écrase le fichier s'il existe (avec confirmation) - Crée les dossiers parents si nécessaire

**Edit : Modifier une partie d'un fichier**

```
Vous : "Ajoute la gestion d'erreur dans la fonction fetchData"

Claude utilise Edit pour modifier uniquement la partie concernée.
```

**Avantages de Edit vs Write :** - Modification chirurgicale - Préserve le reste du fichier - Diff clair des changements - Moins de risque d'erreur

**NotebookEdit : Éditer des notebooks Jupyter**

```
Vous : "Modifie la cellule 3 pour utiliser pandas au lieu de numpy"

Claude utilise NotebookEdit pour modifier le notebook.
```

**Bash : Exécuter des commandes terminal**

```
Vous : "Lance les tests"

Claude utilise Bash avec "npm test" ou "go test ./..."
```

**Commandes typiques :** - `git status`, `git diff`, `git commit` - `npm install`, `npm run build`, `npm test` - `go build`, `go test`, `go mod tidy` - `docker build`, `docker-compose up`

### 4.4 Tools d'orchestration

**Task : Lancer un sous-agent**

```
Vous : "Explore ce projet et fais-moi un rapport"

Claude utilise Task pour lancer un agent Explore.
```

Les subagents disponibles : - **Explore** : Exploration approfondie - **Plan** : Planification d'implémentation - **Bash** : Tâches shell complexes

**AskUserQuestion : Poser une question**

```
Claude : "Je vois deux approches possibles :
1. Utiliser une regex
2. Utiliser une librairie de validation

Laquelle préférez-vous ?"
```

Utilisé quand Claude a besoin de clarification.

**ToolSearch : Rechercher des outils MCP** Permet de trouver des outils additionnels fournis par les serveurs MCP.

### 4.5 Permissions et confirmations

**Système de confirmation :**

Quand Claude veut exécuter une action, vous voyez :

```
Claude wants to run: npm install lodash

[A]ccept  [R]eject  [E]dit  [Always allow]
```

**Options :** - **Accept (a)** : Exécuter cette fois - **Reject (r)** : Refuser - **Edit (e)** : Modifier la commande - **Always allow** : Ne plus demander pour ce type

**Ce que Claude Code ne peut PAS faire :** - Accéder à des fichiers hors du projet (sauf si explicitement autorisé) - Exécuter des commandes root/sudo (sauf autorisation) - Envoyer des données à des tiers - Modifier les fichiers système

**Exercice pratique :**

1. Demandez à Claude de lister tous les fichiers TypeScript
2. Demandez de trouver où une fonction est utilisée
3. Demandez de modifier un fichier et observez le diff
4. Demandez de lancer une commande et notez la demande de permission

---

## Module 5 : Slash Commands natives (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'utiliser les slash commands natives pour gérer sa session (`/help`, `/clear`, `/status`, `/cost`), le contexte (`/context`, `/memory`, `/compact`), et les workflows Git (`/commit`, `/init`).

### 5.1 Commandes de session

**`/help` : Aide et documentation**

```
> /help

Affiche toutes les commandes disponibles et leur description.
```

**`/clear` : Effacer la conversation**

```
> /clear

Efface l'historique de la conversation actuelle.
Utile pour repartir à zéro sans quitter Claude.
```

**`/compact [instructions]` : Compresser le contexte**

```
> /compact

Compresse la conversation pour libérer de l'espace contexte.

> /compact focus on the authentication code

Compresse en gardant le focus sur l'authentification.
```

**`/status` : État de la session**

```
> /status

Affiche :
- Version de Claude Code
- Modèle actuel
- Compte connecté
- Espace de travail
```

**`/cost` : Statistiques d'utilisation**

```
> /cost

Affiche :
- Tokens utilisés (input/output)
- Coût estimé de la session
- Historique d'utilisation
```

**`/exit` : Quitter la session**

```
> /exit

Sauvegarde la session et quitte proprement.
Équivalent à Ctrl+D.
```

### 5.2 Commandes de contexte

**`/context` : Visualiser l'utilisation du contexte**

```
> /context

Affiche :
- Taille de la fenêtre de contexte
- Pourcentage utilisé
- Principaux consommateurs de tokens
```

**`/memory` : Éditer les fichiers CLAUDE.md**

```
> /memory

Ouvre un éditeur pour modifier CLAUDE.md.
Les changements sont appliqués immédiatement.
```

**`/add-dir` : Ajouter des répertoires de travail**

```
> /add-dir ../autre-projet

Ajoute un dossier au contexte de travail.
Permet de travailler sur plusieurs projets liés.
```

### 5.3 Commandes de workflow

**`/commit` : Créer un commit Git**

```
> /commit

Claude va :
1. Analyser les changements (git diff)
2. Proposer un message de commit
3. Créer le commit après confirmation
```

**`/init` : Initialiser un projet avec CLAUDE.md**

```
> /init

Claude crée un fichier CLAUDE.md avec :
- Description du projet
- Stack technique détectée
- Conventions observées
```

**`/plan` : Entrer en mode planification**

```
> /plan

Active le mode plan où Claude :
- Analyse sans modifier
- Propose une stratégie
- Attend validation avant d'agir
```

**`/todos` : Lister les tâches TODO**

```
> /todos

Recherche et affiche tous les TODO/FIXME/HACK
dans le projet avec leur contexte.
```

### 5.4 Commandes de configuration

**`/config` : Ouvrir les paramètres**

```
> /config

Ouvre les paramètres de Claude Code.
Permet de modifier les préférences.
```

**`/model` : Changer de modèle**

```
> /model opus

Change le modèle pour la suite de la session.
Options : sonnet, opus, haiku
```

**`/permissions` : Gérer les autorisations**

```
> /permissions

Affiche et modifie les règles de permission :
- Ce qui est auto-accepté
- Ce qui est toujours refusé
- Ce qui demande confirmation
```

**`/theme` : Changer le thème de couleurs**

```
> /theme dark
> /theme light
> /theme monokai
```

### 5.5 Commandes avancées

**`/resume [session]` : Reprendre une session**

```
> /resume

Affiche les sessions récentes et permet d'en choisir une.

> /resume refactoring-auth

Reprend la session nommée "refactoring-auth".
```

**`/rename <nom>` : Renommer la session**

```
> /rename migration-v2

Renomme la session actuelle pour la retrouver facilement.
```

**`/export [fichier]` : Exporter la conversation**

```
> /export

Exporte vers stdout.

> /export ./session.md

Exporte vers un fichier markdown.
```

**`/rewind` : Revenir à un état précédent**

```
> /rewind

Affiche les checkpoints et permet de revenir en arrière.
Utile si Claude a fait une erreur.
```

**`/doctor` : Diagnostiquer l'installation**

```
> /doctor

Vérifie :
- Configuration correcte
- Connexion API
- Serveurs MCP
- Permissions fichiers
```

**Exercice pratique :**

1. Utilisez `/status` pour voir votre configuration

2. Utilisez `/cost` pour voir l'utilisation
3. Essayez `/init` sur un nouveau projet
4. Utilisez `/commit` après avoir fait une modification
5. Exportez votre conversation avec `/export`

---

## Module 6 : Raccourcis clavier (20 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'utiliser les raccourcis clavier essentiels pour contrôler Claude Code (`Ctrl+C`, `Ctrl+D`, `Esc+Esc`), éditer ses prompts (multiline), et naviguer rapidement (`Shift+Tab`, `Alt+P`).

### 6.1 Contrôles généraux

| Raccourci | Action |
|---|---|
| `Ctrl+C` | Annuler l'opération en cours |
| `Ctrl+D` | Quitter la session |
| `Ctrl+L` | Effacer l'écran terminal |
| `Ctrl+O` | Basculer le mode verbose (voir les détails) |
| `Esc` + `Esc` | Rewind rapide (revenir en arrière) |

**`Ctrl+C` - Annuler :** Interrompt ce que Claude est en train de faire. Utile si : - Claude part dans une mauvaise direction - Une opération prend trop de temps - Vous voulez reformuler votre demande

**`Ctrl+O` - Mode verbose :** Active/désactive l'affichage détaillé des opérations. En mode verbose, vous voyez : - Les outils appelés - Les paramètres utilisés - Les temps d'exécution

### 6.2 Édition de texte

| Raccourci | Action |
|---|---|
| `Ctrl+K` | Supprimer du curseur jusqu'à la fin de ligne |
| `Ctrl+U` | Supprimer la ligne entière |
| `\` + `Enter` | Saut de ligne (multiline) |
| `Option+Enter` | Saut de ligne (macOS) |

**Écrire des prompts multilignes :**

```
> Voici mon prompt sur \
  plusieurs lignes \
  qui forme un seul message
```

Ou utilisez `Option+Enter` (Mac) / `Alt+Enter` (Linux/Windows).

### 6.3 Navigation et modèles

| Raccourci | Action |
|---|---|
| `Shift+Tab` | Basculer les modes de permission |
| `Option+P` / `Alt+P` | Changer de modèle rapidement |
| `Option+T` / `Alt+T` | Basculer Extended Thinking |

| Raccourci | Action |
|---|---|
| `Ctrl+B` | Voir les tâches en arrière-plan |

**`Shift+Tab` - Modes de permission :** Cycle entre : default □ acceptEdits □ plan □ default

**`Option+P` - Changer de modèle :** Cycle entre : sonnet □ opus □ haiku □ sonnet

### 6.4 Raccourcis spéciaux

| Raccourci | Action |
|---|---|
| `/` | Démarrer une slash command |
| `!` | Mode Bash direct |
| `@` | Autocomplete fichiers |
| `Tab` | Accepter les suggestions |

**`!` - Mode Bash direct :**

```
> !git status
```

Exécute directement la commande sans demander à Claude.

**`@` - Référencer des fichiers :**

```
> Regarde @src/api/auth.ts et optimise les imports
```

L'autocomplétion propose les fichiers du projet.

**Exercice pratique :**

1. Pratiquez `Ctrl+C` pour interrompre Claude
2. Écrivez un prompt multiline avec `\` + `Enter`
3. Changez de modèle avec `Option+P`
4. Utilisez `@` pour référencer un fichier
5. Essayez le mode Bash direct avec `!ls -la`

---

## Module 7 : Ajouter du contexte à Claude Code (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de fournir du contexte efficacement à Claude Code via les @mentions, le copier-coller, les images et URLs, et de créer un fichier CLAUDE.md pour persister les informations projet.

### 7.1 Pourquoi le contexte est crucial

Claude Code ne connaît pas votre projet par défaut. Il doit : - Lire les fichiers pour comprendre - Inférer les conventions - Deviner votre stack technique

**Plus de contexte = meilleures réponses**

| Contexte fourni | Qualité réponse |
|---|---|
| "Corrige ce bug" | □ Vague |
| "Corrige le bug dans auth.ts" | □ □ Mieux |
| "Corrige le bug ligne 45 de auth.ts, voici l'erreur: …" | □ □ □ Bon |

| Contexte fourni | Qualité réponse |
|---|---|
| Contexte + CLAUDE.md + stack connue | ☐ ☐ ☐ ☐ ☐ Excellent |

**Équilibre contexte / coût :** - Plus de contexte = plus de tokens = plus cher - Trop de contexte = noise, réponses moins précises - Juste assez de contexte = optimal

### 7.2 Méthodes pour ajouter du contexte

**@mentions : Référencer des fichiers spécifiques**

```
> Regarde @src/api/auth.ts et @src/types/user.ts
> puis explique le flow d'authentification
```

L'autocomplétion aide à trouver les fichiers.

**Copier-coller : Code ou logs**

```
> Voici l'erreur que j'obtiens :

TypeError: Cannot read properties of undefined (reading 'map')
    at UserList.tsx:24
    at Array.map (<anonymous>)
    ...

Trouve et corrige le problème.
```

**Images : Screenshots** Collez directement une image dans le terminal : - Screenshots d'erreurs - Maquettes UI - Diagrammes d'architecture

Claude peut analyser les images et comprendre le contexte visuel.

**URLs : Documentation**

```
> Voici la documentation de l'API que je dois intégrer :
> https://api.example.com/docs

> Implémente un client Go pour cette API.
```

### 7.3 Contexte persistant avec CLAUDE.md

Le fichier `CLAUDE.md` est la **mémoire projet** de Claude Code.

**Créer un CLAUDE.md :**

```
> /init
```

Claude analyse votre projet et crée un CLAUDE.md initial.

**Structure recommandée :**

```
# CLAUDE.md

## Projet
Application de gestion de tâches

## Stack technique
```

```
- Backend: Go 1.21, Chi router, sqlc
- Frontend: Angular 17, Tailwind CSS
- Database: PostgreSQL 15
- Déploiement: Azure Container Apps

## Conventions
- Nommage Go : camelCase pour privé, PascalCase pour exporté
- Tests : fichiers *_test.go dans le même package
- API : REST, versioning via path (/api/v1/...)

## Architecture
/cmd/api        → Point d'entrée
/internal/api   → Handlers HTTP
/internal/db    → Couche base de données
/internal/domain → Logique métier

## Commandes utiles
- `make dev` : Lancer en mode développement
- `make test` : Lancer les tests
- `make build` : Compiler pour production
```

**Le CLAUDE.md est chargé automatiquement** à chaque session.

### 7.4 Ajouter des répertoires de travail

**Via slash command :**

```
> /add-dir ../shared-library
```

**Via flag au lancement :**

```
claude --add-dir ../shared-library --add-dir ../common-types
```

**Cas d'usage :** - **Monorepos** : Travailler sur plusieurs packages - **Projets liés** : Frontend + Backend dans des dossiers séparés - **Librairies partagées** : Voir les types communs

**Exercice pratique :**

1. Créez un CLAUDE.md pour un de vos projets avec `/init`
2. Personnalisez-le avec votre stack et conventions
3. Utilisez @ pour référencer des fichiers spécifiques
4. Si vous avez un monorepo, utilisez `/add-dir`

---

## Module 8 : Workflow quotidien (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'utiliser Claude Code pour explorer une codebase, modifier du code, créer des commits Git, débugger des erreurs, écrire des tests et générer de la documentation.

### 8.1 Explorer une codebase

**Questions d'exploration efficaces :**

```
> Décris la structure de ce projet
```

```
> Comment fonctionne l'authentification ?

> Où sont gérées les erreurs API ?

> Montre-moi le flow de données pour créer un utilisateur

> Quelles sont les dépendances principales et pourquoi ?
```

**Technique : Du général au spécifique**

1. Vue d'ensemble : "Décris ce projet"
2. Architecture : "Explique l'architecture"
3. Composant : "Comment fonctionne le module X ?"
4. Détail : "Explique cette fonction ligne par ligne"

### 8.2 Modifier du code

**Corrections simples :**

```
> Dans @src/utils.ts, la fonction formatDate
> ne gère pas les fuseaux horaires. Corrige ça.
```

**Ajout de fonctionnalité :**

```
> Ajoute une fonction de validation d'email
> dans @src/validators.ts qui :
> - Vérifie le format basique
> - Vérifie que le domaine existe
> - Retourne un objet {valid: boolean, error?: string}
```

**Refactoring guidé :**

```
> Cette fonction fait 200 lignes. Refactorise-la en :
> - Une fonction principale de coordination
> - Des sous-fonctions avec responsabilité unique
> - Garde la même API publique
```

### 8.3 Intégration Git

**Voir le status :**

```
> Montre-moi ce qui a changé depuis le dernier commit
```

**Créer des commits :**

```
> /commit
```

Claude va : 1. Analyser `git status` et `git diff` 2. Regarder les commits récents pour le style 3. Proposer un message de commit approprié 4. Créer le commit après votre validation

**Message de commit généré :**

```
feat(auth): add JWT token refresh mechanism

- Add refreshToken endpoint in auth handler
- Implement token rotation for security
- Add refresh token to user session

Closes #123
```

### 8.4 Débugger efficacement

**Fournir le contexte complet :**

```
> J'ai cette erreur quand je clique sur "Sauvegarder" :

[Coller l'erreur complète]

Le comportement attendu est : sauvegarder l'utilisateur et rediriger
Le comportement actuel est : erreur 500

Fichiers concernés probables : @src/api/users.ts @src/db/users.ts

Trouve la cause et corrige.
```

**Laisser Claude investiguer :**

Claude va : 1. Lire les fichiers mentionnés 2. Tracer le chemin de l'erreur 3. Identifier la cause racine 4. Proposer une correction 5. Optionnellement lancer les tests

### 8.5 Écrire des tests

**Générer des tests unitaires :**

```
> Génère les tests unitaires pour @src/services/user.service.ts
> Utilise Jest et mock la base de données
```

**Couvrir les cas limites :**

```
> Ajoute des tests pour les cas limites :
> - Input vide
> - Caractères spéciaux
> - Valeurs null/undefined
> - Dépassement de limites
```

**TDD assisté :**

```
> Je veux implémenter une fonction de recherche floue.
> Écris d'abord les tests (TDD), puis l'implémentation.
```

### 8.6 Documenter

**Générer de la documentation :**

```
> Génère la documentation JSDoc pour toutes les fonctions
> exportées de @src/api/
```

**Créer un README :**

```
> Crée un README.md pour ce projet avec :
> - Description
> - Installation
> - Usage
> - API endpoints
> - Contribution guidelines
```

**Exercice pratique : Débugger un projet**

1. Clonez un projet avec un bug intentionnel (ou créez-en un)
2. Lancez Claude Code
3. Décrivez le symptôme du bug

4. Laissez Claude investiguer et corriger
5. Validez la correction avec les tests

---

## Module 9 : Subagents - Introduction (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'expliquer le rôle des subagents (Explore, Plan, Bash), de reconnaître quand Claude Code en lance un, et de comprendre les résultats retournés.

### 9.1 Qu'est-ce qu'un Subagent ?

Un Subagent est un **agent spécialisé** que Claude Code peut lancer pour des tâches spécifiques.

**Analogie :** - Claude Code = Chef de projet - Subagents = Spécialistes (explorateur, architecte, ops)

**Pourquoi des subagents ?** - Tâches qui nécessitent beaucoup de contexte - Spécialisation (exploration vs planification) - Isolation (ne pollue pas la conversation principale)

### 9.2 Subagents intégrés

**Explore : Explorer et comprendre**

```
> Explore ce projet et fais un rapport sur l'architecture
```

L'agent Explore va : - Parcourir tous les fichiers - Identifier les patterns - Cartographier les dépendances - Produire un rapport structuré

**Cas d'usage :** - Nouveau sur un projet - Comprendre du code legacy - Audit d'architecture

**Plan : Planifier une implémentation**

```
> /plan Implémenter un système de notifications push
```

L'agent Plan va : - Analyser les besoins - Identifier les composants concernés - Proposer une architecture - Découper en tâches - Estimer la complexité

**Cas d'usage :** - Nouvelles features complexes - Refactoring majeur - Migration technique

**Bash : Exécuter des commandes**

```
> Configure l'environnement de dev avec Docker
```

L'agent Bash est spécialisé dans : - Commandes shell complexes - Scripts de setup - Opérations système

**general-purpose : Tâches variées** Agent polyvalent pour des tâches qui ne rentrent pas dans les autres catégories.

### 9.3 Observer les Subagents

**Reconnaître un subagent :**

```
┌─ Task: Exploring codebase ──────────────────┐
│                                             │
│ Reading src/index.ts...                     │
│ Analyzing structure...                      │
│                                             │
```

└──────────────────────────────────────────────────────────────────────┘

L'interface indique clairement quand un subagent travaille.

**Résultat d'un subagent :**

Le subagent retourne un résumé à l'agent principal. Vous ne voyez que : - Le résultat final - Les actions importantes - Les fichiers modifiés

**Parallèle vs Séquentiel :** - Claude peut lancer plusieurs subagents en parallèle - Les tâches indépendantes s'exécutent simultanément - Les tâches dépendantes s'exécutent séquentiellement

**Exercice pratique :**

1. Demandez à Claude d'explorer un projet inconnu
2. Observez le subagent Explore en action
3. Demandez de planifier une feature complexe
4. Observez la différence entre Explore et Plan

---

## Module 10 : Skills et Plugins - Introduction (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de distinguer les slash commands des skills, d'utiliser des skills existantes, et de comprendre le concept de plugin comme bundle de skills, hooks et MCP servers.

### 10.1 Qu'est-ce qu'une Skill ?

Une Skill est un **prompt packagé** réutilisable. C'est comme une macro ou un template.

**Différence Slash Command vs Skill :**

| Aspect | Slash Command | Skill |
|---|---|---|
| Source | Native Claude Code | Plugins/Utilisateur |
| Exemple | `/help`, `/commit` | `/code-review`, `/standup` |
| Modifiable | Non | Oui |
| Partageable | Non | Oui |

**Syntaxe d'appel :**

```
> /skill-name
```

ou

```
> /plugin-name:skill-name
```

### 10.2 Utiliser une Skill existante

**Voir les skills disponibles :**

```
> /help skills
```

**Skills populaires de la communauté :**

| Skill | Description |
|---|---|
| `/code-review` | Revue de code automatisée |

| Skill | Description |
|---|---|
| `/standup` | Générer un rapport de standup |
| `/changelog` | Générer un changelog |
| `/refactor` | Guide de refactoring |
| `/security-audit` | Audit de sécurité basique |

### 10.3 Qu'est-ce qu'un Plugin ?

Un Plugin est un **bundle** qui peut contenir : - Des skills - Des hooks (automatisations) - Des serveurs MCP - Des configurations

**Marketplace officiel :**

```
# Voir les plugins disponibles
claude plugins list

# Installer un plugin
claude plugins install plugin-name
```

### 10.4 Bonnes pratiques débutant

**Quand utiliser Claude Code vs coder soi-même :**

**Utiliser Claude Code pour :** - Explorer du code inconnu - Générer du code boilerplate - Écrire des tests - Refactoring mécanique - Documentation

**Vérifier attentivement :** - Code de sécurité (auth, crypto) - Logique métier critique - Code de performance

**Coder soi-même :** - Algorithmes très spécifiques - Code ultra-optimisé - Décisions d'architecture majeures

**Toujours vérifier le code généré :**

1. Lire le diff avant d'accepter
2. Lancer les tests après modification
3. Review comme du code d'un collègue

**Exercice pratique :**

1. Listez les skills disponibles
2. Essayez `/commit` sur un projet avec des changements
3. Si disponible, essayez une skill de la communauté

---

## Module 11 : MCP Servers - Introduction (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'expliquer le Model Context Protocol (MCP), d'installer un serveur MCP simple avec `claude mcp add`, et d'utiliser les outils fournis par les serveurs installés.

### 11.1 Qu'est-ce qu'un MCP Server ?

MCP = **Model Context Protocol**

C'est un protocole standard qui permet d'étendre Claude Code avec de nouveaux outils.

**Analogie :** - Claude Code = Téléphone - MCP Servers = Applications - Outils MCP = Fonctionnalités des apps

**Ce que les MCP Servers permettent :** - Accès aux bases de données - Intégration GitHub/GitLab - Automatisation navigateur - Accès à des APIs tierces

### 11.2 MCP Servers pré-installés

**Voir les serveurs actifs :**

```
> /mcp
```

**Serveurs courants :**

| Serveur | Outils fournis |
|---|---|
| Chrome DevTools | Navigation, screenshots, formulaires |
| Filesystem | Accès fichiers étendu |
| Git | Opérations git avancées |

### 11.3 Installer un MCP Server simple

**Ajouter un serveur :**

```
claude mcp add filesystem
```

**Lister les serveurs installés :**

```
claude mcp list
```

**Exemple : Serveur PostgreSQL**

```
# Installer le serveur PostgreSQL
claude mcp add postgres

# Configurer la connexion
# Dans ~/.claude/mcp.json ou .mcp.json du projet
```

### 11.4 Utiliser les outils MCP

Une fois installés, les outils MCP apparaissent automatiquement.

**Exemple avec PostgreSQL :**

```
> Montre-moi les utilisateurs qui se sont inscrits ce mois

Claude utilise l'outil MCP postgres_query avec :
SELECT * FROM users
WHERE created_at >= date_trunc('month', current_date)
```

**Permissions :** - Les serveurs MCP demandent des permissions - Certains nécessitent une authentification OAuth - Les credentials sont stockés de manière sécurisée

**Exercice pratique :**

1. Utilisez `/mcp` pour voir les serveurs actifs
2. Installez un serveur MCP simple (filesystem)
3. Testez un outil du serveur installé

---

## Module 12 : Gestion des sessions (20 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de gérer ses sessions Claude Code (reprendre, nommer, exporter), de naviguer dans l'historique, et d'utiliser les checkpoints pour revenir à un état précédent avec `/rewind`.

### 12.1 Sessions et persistance

**Les sessions sont sauvegardées automatiquement.**

Chaque session contient : - L'historique de conversation - Les fichiers lus - Les actions effectuées - Les checkpoints

**Reprendre la dernière session :**

```
claude -c
# ou
claude --continue
```

**Reprendre une session spécifique :**

```
claude -r nom-session
# ou
claude --resume nom-session
```

**Nommer une session :**

```
> /rename feature-auth-sso
```

### 12.2 Historique et navigation

**Voir les sessions passées :**

```
> /resume
```

Affiche une liste des sessions récentes avec : - Nom/ID - Date - Projet - Résumé

**Exporter une conversation :**

```
> /export ./ma-session.md
```

Crée un fichier markdown avec tout l'historique.

### 12.3 Checkpoints et Rewind

**Checkpoints automatiques :**

Claude Code sauvegarde des "checkpoints" : - Avant chaque modification de fichier - Avant chaque commande destructive - À intervalles réguliers

**Revenir en arrière :**

```
> /rewind
```

Affiche les checkpoints et permet de choisir.

**Rewind rapide :**

`Esc` + `Esc` → Revient au checkpoint précédent

**Limitations :** - Ne peut pas annuler les changements externes (autre éditeur, git pull) - Les commandes système ne sont pas réversibles

**Exercice pratique :**

1. Travaillez sur un projet pendant 10 minutes
2. Nommez votre session avec `/rename`
3. Quittez avec `/exit`
4. Reprenez avec `claude -r nom-session`
5. Essayez `/rewind` pour voir les checkpoints

---

## Projet de fin de partie gratuite : BizCraft Diagnostic (Phase 0)

### Contexte du projet

**BizCraft** est un SaaS qui aide les entrepreneurs à structurer leur entreprise selon les principes du livre *The E-Myth Revisited* de Michael Gerber.

Le concept central : tout entrepreneur a trois personnalités : - **L'Entrepreneur** : Le visionnaire qui voit les opportunités - **Le Manager** : L'organisateur qui crée les systèmes - **Le Technicien** : L'expert qui fait le travail

Le déséquilibre de ces trois profils est souvent la cause des difficultés des petites entreprises.

### Ce qu'on construit

Le **Diagnostic Gratuit** est le tunnel d'acquisition du SaaS. C'est un questionnaire qui : 1. Évalue le profil E/M/T de l'entrepreneur 2. Génère un rapport personnalisé 3. Capture l'email pour le nurturing

### Stack technique

- **Frontend** : Angular 17+ standalone, Tailwind CSS
- **Backend** : Go 1.21+
- **Base de données** : PostgreSQL
- **PDF** : go-wkhtmltopdf ou chromedp
- **Email** : SMTP

### Étapes de construction avec Claude Code

#### Étape 1 : Setup du projet

```
> Initialise un nouveau projet avec :
> - Backend Go avec Chi router
> - Frontend Angular 17 standalone
> - Docker Compose pour PostgreSQL
> - Structure de dossiers propre
```

#### Étape 2 : Créer le CLAUDE.md

```
> /init

Puis personnaliser avec les conventions BizCraft.
```

#### Étape 3 : Frontend - Landing page

```
> Crée une landing page /diagnostic avec :
> - Hero section avec CTA "Faire le diagnostic"
> - 3 sections expliquant E/M/T
> - Témoignages
> - Footer avec mentions légales
> Utilise Tailwind CSS.
```

#### Étape 4 : Frontend - Questionnaire

```
> Crée un composant de questionnaire avec :
> - 20 questions à choix multiples
> - Barre de progression
> - Navigation avant/arrière
> - Sauvegarde des réponses en state
> - Soumission à l'API
```

**Étape 5 : Backend - API de soumission**

```
> Crée l'endpoint POST /api/diagnostic/submit qui :
> - Reçoit les réponses du questionnaire
> - Calcule le score E/M/T
> - Sauvegarde en base
> - Retourne les scores
```

**Étape 6 : Algorithme de scoring**

```
> Implémente l'algorithme de scoring :
> - Chaque question a un poids pour E, M, et T
> - Calcule le pourcentage pour chaque profil
> - Le total fait 100%
```

**Étape 7 : Génération PDF**

```
> Crée la génération du rapport PDF avec :
> - En-tête avec logo BizCraft
> - Radar chart des scores
> - Interprétation personnalisée
> - Conseils selon le profil dominant
```

**Étape 8 : Envoi email**

```
> Ajoute l'envoi d'email avec :
> - Template HTML responsive
> - PDF en pièce jointe
> - Configuration SMTP via variables d'environnement
```

**Livrables attendus**

- ☐ Projet initialisé avec structure correcte
- ☐ CLAUDE.md personnalisé pour BizCraft
- ☐ Landing page fonctionnelle
- ☐ Questionnaire interactif complet
- ☐ API de soumission avec scoring
- ☐ Génération PDF du rapport
- ☐ Envoi d'email automatique
- ☐ Tests unitaires backend
- ☐ README avec instructions

---

# PARTIE 2 : MAÎTRISE AVANCÉE

**Objectif de la Partie 2**

À l'issue de cette partie, l'apprenant sera capable de : - Configurer Claude Code de manière avancée (settings.json, variables d'environnement, managed settings) - Créer des fichiers CLAUDE.md modulaires avec imports - Gérer le contexte efficacement et optimiser l'usage des tokens - Configurer des règles de

permission granulaires et sécurisées - Installer et créer des serveurs MCP (PostgreSQL, GitHub, Chrome DevTools) - Créer des skills et subagents personnalisés - Implémenter des hooks d'automatisation (pre-commit, formatage, notifications) - Intégrer Claude Code dans VS Code, JetBrains et les workflows CI/CD - Déployer Claude Code en contexte enterprise (managed settings, providers cloud)

---

## Module 13 : Configuration avancée - Fichiers (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de naviguer la hiérarchie des fichiers de configuration (managed, local, project, user), de créer un settings.json personnalisé, et d'utiliser les fichiers spéciaux (.claudeignore, .mcp.json).

### 13.1 Hiérarchie des fichiers de configuration

Claude Code utilise une hiérarchie de configuration avec différents niveaux de priorité :

```
┌─────────────────────────────────────────────┐
│ PRIORITÉ LA PLUS HAUTE                      │
├─────────────────────────────────────────────┤
│ 1. Managed Settings (IT/entreprise)         │
│    └── managed-settings.json                │
├─────────────────────────────────────────────┤
│ 2. Flags CLI                                │
│    └── claude --model opus                  │
├─────────────────────────────────────────────┤
│ 3. Local Settings (personnel, gitignored)   │
│    └── .claude/*.local.*                    │
├─────────────────────────────────────────────┤
│ 4. Project Settings (partagé équipe)        │
│    └── .claude/settings.json                │
├─────────────────────────────────────────────┤
│ 5. User Settings (tous projets)             │
│    └── ~/.claude/settings.json              │
│                                             │
│ PRIORITÉ LA PLUS BASSE                      │
└─────────────────────────────────────────────┘
```

**Managed (Entreprise) :** - Fichier `managed-settings.json` - Contrôlé par IT/administrateurs - Ne peut pas être overridé par l'utilisateur - Utilisé pour politiques d'entreprise

**Local (Personnel) :** - Dossier `.claude/` avec fichiers `*.local.*` - Automatiquement dans `.gitignore` - Vos préférences personnelles - API keys, tokens personnels

**Project (Équipe) :** - Dossier `.claude/` sans `.local` - Versionné avec le projet - Partagé avec l'équipe - Conventions du projet

**User (Global) :** - Dossier `~/.claude/` - Appliqué à tous vos projets - Vos préférences par défaut

### 13.2 Le fichier settings.json

**Structure complète :**

```
{
  "model": "sonnet",
```

```
  "customApiKeyHelper": null,

  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(go test *)",
      "Read(*)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Write(.env*)"
    ],
    "ask": [
      "Bash(git push *)"
    ]
  },

  "env": {
    "NODE_ENV": "development",
    "CUSTOM_VAR": "value"
  },

  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://..."
      }
    }
  },

  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "command": ["npm", "run", "format"]
      }
    ]
  }
}
```

**Options de modèle :**

```
{
  "model": "sonnet",          // Modèle par défaut
  "fallbackModel": "haiku",   // Si le principal échoue
  "maxThinkingTokens": 10000  // Pour Extended Thinking
}
```

**Options d'interface :**

```
{
  "theme": "dark",
  "verbose": false,
```

```
  "notifications": true,
  "autoCompactThreshold": 0.8
}
```

### 13.3 Fichiers spéciaux

**.claudeignore** Fonctionne comme `.gitignore` pour exclure des fichiers du contexte :

```
# Dépendances
node_modules/
vendor/

# Build
dist/
build/

# Secrets
.env*
*.key
*.pem

# Gros fichiers
*.zip
*.tar.gz

# Fichiers générés
coverage/
*.log
```

**.mcp.json** Configuration des serveurs MCP au niveau projet :

```
{
  "servers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

**~/.claude.json** Configuration utilisateur globale :

```
{
  "oauthTokens": { ... },
  "mcpServers": { ... },
  "defaultModel": "sonnet",
  "theme": "dark"
}
```

### 13.4 Configuration par environnement

**Variables d'environnement importantes :**

| Variable | Description |
|---|---|
| `ANTHROPIC_API_KEY` | Clé API Anthropic |
| `ANTHROPIC_MODEL` | Modèle par défaut |
| `CLAUDE_PROJECT_DIR` | Dossier de travail |
| `CLAUDE_CONFIG_DIR` | Dossier de config |

**Variables de contexte :**

| Variable | Description |
|---|---|
| `CLAUDE_MAX_TOKENS` | Limite de tokens |
| `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` | Seuil de compaction auto |
| `MAX_THINKING_TOKENS` | Tokens pour Extended Thinking |

**Variables de debugging :**

| Variable | Description |
|---|---|
| `CLAUDE_DEBUG` | Active les logs de debug |
| `CLAUDE_VERBOSE` | Mode verbose par défaut |

**Exemple de configuration shell :**

```
# ~/.zshrc ou ~/.bashrc

export ANTHROPIC_API_KEY="sk-ant-..."
export ANTHROPIC_MODEL="sonnet"
export CLAUDE_AUTOCOMPACT_PCT_OVERRIDE="0.7"

# Alias pratiques
alias cc='claude'
alias ccc='claude -c'
alias ccp='claude -p'
```

**Exercice :**

1. Créez un fichier `.claude/settings.json` dans un projet
2. Configurez des permissions personnalisées
3. Ajoutez des variables d'environnement
4. Créez un `.claudeignore` pour exclure les fichiers inutiles

---

## Module 14 : CLAUDE.md et Memory System (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de créer un CLAUDE.md structuré et adapté à son projet, d'utiliser la hiérarchie des fichiers mémoire (global, projet, local), et de modulariser avec la syntaxe d'import.

### 14.1 Le fichier CLAUDE.md

CLAUDE.md est la **mémoire projet** de Claude Code. Il est chargé automatiquement à chaque session.

**Contenu recommandé :**

```
# CLAUDE.md

## À propos du projet
[Description courte du projet et son objectif]

## Stack technique
- Backend: [langages, frameworks, versions]
- Frontend: [langages, frameworks, versions]
- Database: [type, version]
- Infrastructure: [cloud, containers, etc.]

## Architecture
[Description de l'organisation du code]

## Conventions de code
[Nommage, formatage, patterns utilisés]

## Commandes de développement
- `make dev` : [description]
- `make test` : [description]
- `make build` : [description]

## Points d'attention
[Choses que Claude doit savoir : legacy, dette technique, etc.]
```

**Exemple pour un projet Go/Angular :**

```
# CLAUDE.md

## Projet
API de gestion de tâches avec frontend Angular.

## Stack
- Backend: Go 1.22, Chi router, sqlc pour queries
- Frontend: Angular 18, Tailwind CSS, standalone components
- DB: PostgreSQL 16
- Deploy: Azure Container Apps

## Architecture

### Backend (/api)
```

/cmd/api □ Point d'entrée /internal/ /handlers □ HTTP handlers (thin layer) /services □ Business logic /repository □ Data access (sqlc generated) /domain □ Entities et value objects /migrations □ SQL migra-

tions

```
### Frontend (/web)
```

/src/app/ /core □ Services singleton, guards /shared □ Components réutilisables /features □ Feature modules /pages □ Pages/routes principales

```
## Conventions

### Go
- Erreurs: wrap avec fmt.Errorf("context: %w", err)
- Tests: fichiers _test.go, mocks avec gomock
- Logs: slog structured logging

### Angular
- Components: standalone, OnPush change detection
- State: signals pour state local, services pour partagé
- Forms: reactive forms uniquement

## Commandes
- `make dev` : Lance API + DB + Frontend
- `make test` : Tests Go + Angular
- `make migrate` : Applique les migrations
- `make generate` : Génère le code sqlc
```

**14.2 Hiérarchie des fichiers mémoire**

```
┌──────────────────────────────────────────┐
│ Chargement automatique (du plus au moins  │
│ spécifique) :                             │
├──────────────────────────────────────────┤
│                                          │
│ ~/.claude/CLAUDE.md                      │
│ └── Global : s'applique à TOUS vos projets │
│     Ex: vos préférences personnelles     │
│                                          │
│ .claude/CLAUDE.md                        │
│ └── Projet : partagé avec l'équipe       │
│     Ex: conventions du projet            │
│                                          │
│ .claude/CLAUDE.local.md                  │
│ └── Personnel : gitignored               │
│     Ex: notes personnelles, TODO perso   │
│                                          │
│ CLAUDE.md (racine)                       │
│ └── Alternatif : si pas de dossier .claude │
│                                          │
└──────────────────────────────────────────┘
```

**Cas d'usage par niveau :**

| Niveau | Quoi mettre |
|---|---|
| Global (~/.claude/CLAUDE.md) | Style de code préféré, langues, habitudes |
| Projet (.claude/CLAUDE.md) | Stack, architecture, conventions équipe |
| Local (.claude/CLAUDE.local.md) | Notes perso, TODO, expériences en cours |

### 14.3 Imports et modularité

Pour les gros projets, découpez le CLAUDE.md en fichiers modulaires.

**Syntaxe d'import :**

```
# CLAUDE.md

## Projet
Application BizCraft SaaS

<!-- @import ./rules/api.md -->
<!-- @import ./rules/angular.md -->
<!-- @import ./rules/security.md -->
<!-- @import ./rules/testing.md -->
```

**Structure recommandée :**

```
.claude/
├── CLAUDE.md              # Fichier principal
├── CLAUDE.local.md        # Notes perso (gitignored)
└── rules/
    ├── api.md             # Conventions API Go
    ├── angular.md         # Standards Angular
    ├── security.md        # Règles de sécurité
    ├── testing.md         # Conventions de test
    └── azure.md           # Patterns Azure
```

**Exemple de fichier modulaire (.claude/rules/api.md) :**

```
# Conventions API Go

## Structure des handlers
- Un fichier par ressource : users.go, tasks.go
- Pattern : Handler struct avec dépendances injectées
- Erreurs HTTP centralisées via httputil.Error()

## Validation
- Utiliser go-playground/validator
- Valider en entrée de handler
- Messages d'erreur explicites

## Réponses
- JSON uniquement
- Enveloppe standard : {data, error, meta}
- Codes HTTP corrects (201 create, 204 delete...)

## Logging
- slog pour tous les logs
```

```
- Contexte : request_id, user_id
- Niveaux : Debug dev, Info prod
```

**Patterns glob pour chargement conditionnel :**

```
<!-- @import ./rules/*.md -->
<!-- @import ./rules/api-*.md -->
```

### 14.4 Bonnes pratiques

**DO :** - Garder le CLAUDE.md principal concis (< 500 lignes) - Utiliser les imports pour la modularité - Versionner avec le projet (sauf .local) - Mettre à jour quand les conventions changent - Inclure des exemples de code

**DON'T :** - Dupliquer des informations - Inclure des secrets ou tokens - Écrire un roman (Claude a un contexte limité) - Laisser des informations obsolètes

**Exercice :**

1. Créez un CLAUDE.md modulaire pour votre stack
2. Séparez en fichiers : api.md, frontend.md, testing.md
3. Utilisez les imports dans le fichier principal
4. Ajoutez un CLAUDE.local.md avec vos notes perso

---

## Module 15 : Contrôler le contexte (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de comprendre la fenêtre de contexte et ses limites, d'utiliser `/compact` et `/clear` efficacement, et d'optimiser l'utilisation des tokens pour des sessions longues.

### 15.1 Comprendre la fenêtre de contexte

La "fenêtre de contexte" est la mémoire de travail de Claude.

**Taille selon le modèle :**

| Modèle | Contexte | ~Équivalent |
|---|---|---|
| Haiku | 200K tokens | ~150K mots |
| Sonnet | 200K tokens | ~150K mots |
| Opus | 200K tokens | ~150K mots |

**Ce qui consomme du contexte :**

| Élément | Consommation |
|---|---|
| CLAUDE.md | Chargé en entier |
| Fichiers lus | Contenu complet |
| Historique conversation | Accumule |
| Résultats de commandes | Variable |
| Outils appelés | Paramètres + résultats |

**Visualiser l'usage :**

```
> /context

Context usage: 45,230 / 200,000 tokens (23%)

Top consumers:
- Conversation history: 20,000 tokens
- src/api/users.go: 5,200 tokens
- CLAUDE.md: 3,000 tokens
- Tool results: 17,030 tokens
```

### 15.2 Stratégies de compaction

**Compaction manuelle :**

```
> /compact

Compresse la conversation en gardant les points essentiels.
```

**Compaction avec focus :**

```
> /compact focus on authentication implementation

Compresse en gardant spécifiquement le contexte d'auth.
```

**Auto-compaction :**

Claude Code compacte automatiquement quand le contexte atteint ~80%.

**Configurer le seuil :**

```
# Dans votre shell ou settings.json
export CLAUDE_AUTOCOMPACT_PCT_OVERRIDE="0.7"  # 70%
```

**Quand compacter manuellement :** - Changement de sujet dans la même session - Après une longue exploration - Avant une tâche qui nécessitera beaucoup de fichiers

### 15.3 Optimiser l'utilisation du contexte

**Éviter de charger des fichiers inutiles :**

```
# Mauvais : charge tout
> Montre-moi tous les fichiers du projet

# Bon : charge uniquement ce qui est pertinent
> Montre-moi la structure du dossier src/api/
```

**Utiliser les bons patterns de recherche :**

```
# Mauvais : potentiellement beaucoup de résultats
> Cherche "error" dans tout le projet

# Bon : ciblé
> Cherche "error handling" dans src/api/*.go
```

**Fermer et rouvrir pour les sessions longues :**

Si vous travaillez depuis longtemps sur le même sujet, parfois il vaut mieux : 1. `/export` la conversation 2. Fermer la session 3. Ouvrir une nouvelle session 4. Résumer ce qui a été fait

**Déléguer aux subagents :**

Les subagents ont leur propre contexte. Les opérations verbeuses (exploration, recherche large) sont mieux dans un subagent.

```
> Explore ce projet et résume l'architecture

Le subagent Explore utilise son propre contexte.
Seul le résumé revient dans votre conversation.
```

### 15.4 Gérer les grands projets

**Monorepos :**

```
# Ne pas charger tout le monorepo
cd packages/api
claude

# Ajouter seulement ce qui est nécessaire
> /add-dir ../shared-types
```

**Utiliser /add-dir stratégiquement :**

```
> /add-dir ../shared-types   # Oui : types partagés
> /add-dir ../other-api      # Non : pas pertinent
```

**CLAUDE.md par sous-projet :**

```
monorepo/
├── CLAUDE.md                 # Contexte global monorepo
├── packages/
│   ├── api/
│   │   └── CLAUDE.md         # Spécifique à l'API
│   ├── web/
│   │   └── CLAUDE.md         # Spécifique au frontend
│   └── shared/
│       └── CLAUDE.md         # Spécifique aux types partagés
```

**Exercice :**

1. Utilisez `/context` pour voir votre usage
2. Faites une session avec beaucoup de fichiers
3. Utilisez `/compact` pour réduire
4. Comparez avant/après avec `/context`

---

## Module 16 : Permissions et Sécurité (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de choisir le mode de permission adapté (default, plan, acceptEdits, bypassPermissions), de configurer des règles allow/deny/ask granulaires, et d'implémenter les bonnes pratiques de sécurité.

### 16.1 Modes de permission

Claude Code offre plusieurs niveaux de contrôle sur ce qu'il peut faire.

**Default : Confirmation pour chaque action**

```
Claude wants to edit: src/api/users.go
[A]ccept [R]eject [E]dit [Always]

Claude wants to run: npm test
[A]ccept [R]eject [E]dit [Always]
```

C'est le mode le plus sûr pour les débutants.

**Plan Mode : Affiche sans exécuter**

```
claude --permission-mode plan
```

Claude va : - Analyser et planifier - Montrer ce qu'il ferait - Ne rien exécuter - Attendre votre validation explicite

Idéal pour : comprendre avant d'agir, review de plan.

**Accept Edits : Auto-accepte les fichiers**

```
claude --permission-mode acceptEdits
```

- Modifications de fichiers : automatiques
- Commandes shell : demandent confirmation
- Bon équilibre confiance/sécurité

**Bypass : Aucune confirmation**

```
claude --permission-mode bypass
```

**Attention : Dangereux** - N'utilisez que pour : - Scripts automatisés contrôlés - Environnements sandboxés - CI/CD avec limites strictes

### 16.2 Règles de permission granulaires

**Syntaxe des règles :**

`Tool(pattern)`

**Exemples :**

| Règle | Signification |
|---|---|
| `Bash(npm run *)` | Toutes les commandes npm run |
| `Bash(go test ./...)` | Tests Go uniquement |
| `Read(src/**)` | Lecture dans src/ |
| `Edit(*.test.ts)` | Édition des fichiers de test |
| `Write(*.md)` | Création de fichiers markdown |

**Configuration dans settings.json :**

```
{
  "permissions": {
    "allow": [
      "Read(*)",
      "Bash(npm run *)",
      "Bash(go test *)",
      "Bash(make *)",
      "Edit(src/**)",
      "Edit(tests/**)"
```

```
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(sudo *)",
      "Write(.env*)",
      "Edit(*.key)",
      "Bash(curl * | sh)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(npm publish)",
      "Write(migrations/*)"
    ]
  }
}
```

**Patterns avancés :**

```
{
  "permissions": {
    "allow": [
      "Bash(docker-compose up *)",
      "Bash(docker-compose down)",
      "Edit(src/**/*.{ts,tsx})",
      "Read(**/*.{go,ts,md})"
    ]
  }
}
```

### 16.3 Sandboxing

**Isolation du filesystem :**

Par défaut, Claude Code ne peut accéder qu'au : - Dossier de travail actuel - Dossiers ajoutés avec `--add-dir`

**Isolation réseau :**

Claude Code ne peut pas : - Ouvrir des connexions arbitraires - Exécuter des serveurs écoutant sur des ports - Accéder à des APIs sans votre permission

**Protection contre l'injection de prompts :**

Claude Code détecte et refuse les tentatives d'injection dans : - Contenu de fichiers lus - Résultats de commandes - Réponses d'APIs

**Configuration du sandbox :**

```
{
  "sandbox": {
    "allowedPaths": [
      "/home/user/projects",
      "/tmp"
    ],
    "deniedPaths": [
      "/etc",
      "/var",
```

```
      "~/.ssh"
    ],
    "networkAccess": "restricted"
  }
}
```

### 16.4 Sécurité en équipe

**Managed Settings (Entreprise) :**

L'IT peut déployer un fichier `managed-settings.json` :

**macOS** : `/Library/Application Support/ClaudeCode/managed-settings.json` **Linux** : `/etc/claude-code/managed-settings.json` **Windows** : `%PROGRAMDATA%\ClaudeCode\managed-settings.json`

```
{
  "permissions": {
    "deny": [
      "Bash(curl * | *)",
      "Bash(wget * | *)",
      "Write(*.env*)",
      "Write(*credentials*)"
    ]
  },
  "mcpServers": {
    "allowlist": ["postgres", "github"],
    "denylist": ["*"]
  },
  "model": {
    "allowed": ["sonnet", "haiku"],
    "denied": ["opus"]
  }
}
```

**Audit et logging :**

Activez le logging pour traçabilité :

```
{
  "logging": {
    "enabled": true,
    "level": "info",
    "path": "/var/log/claude-code/",
    "includeToolCalls": true
  }
}
```

**Exercice :**

1. Créez un fichier settings.json avec des permissions personnalisées
2. Autorisez les commandes npm/go que vous utilisez
3. Bloquez les commandes destructives
4. Testez que les permissions fonctionnent

---

## Module 17 : MCP Servers - Maîtrise complète (1h)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'expliquer l'architecture MCP (stdio vs HTTP), d'installer et configurer des serveurs MCP populaires (PostgreSQL, GitHub, Chrome DevTools), et de créer un serveur MCP personnalisé avec le SDK TypeScript.

### 17.1 Architecture MCP

**Model Context Protocol (MCP) :**

MCP est un protocole standardisé qui permet d'étendre les capacités de Claude Code.

```
┌──────────────────────────────────────────────────────┐
│                    CLAUDE CODE                       │
│  ┌──────────────────────────────────────────┐        │
│  │              MCP Client                  │        │
│  │   (parle le protocole MCP)               │        │
│  └──────────────────────────────────────────┘        │
│                 │           │           │            │
│                 ▼           ▼           ▼            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│  │ MCP Server   │  │ MCP Server   │  │ MCP Server   │
│  │ PostgreSQL   │  │   GitHub     │  │   Chrome     │
│  └──────────────┘  └──────────────┘  └──────────────┘
│          │              │              │            │
└──────────┼──────────────┼──────────────┼────────────┘
           ▼              ▼              ▼

     ┌──────────┐    ┌──────────┐    ┌──────────┐
     │ Database │    │  GitHub  │    │ Browser  │
     │          │    │   API    │    │          │
     └──────────┘    └──────────┘    └──────────┘
```

**Communication stdio vs HTTP :**

| Type | Usage | Exemple |
|---|---|---|
| stdio | Serveurs locaux | PostgreSQL local |
| HTTP/SSE | Serveurs distants | API SaaS |

**Serveurs locaux :** - S'exécutent sur votre machine - Communication via stdin/stdout - Plus rapides, pas de réseau

**Serveurs distants :** - Hébergés ailleurs - Communication HTTP/SSE - Nécessitent authentification

### 17.2 Gestion des serveurs MCP

**Ajouter un serveur :**

```
# Serveur npm
claude mcp add postgres

# Serveur avec configuration
claude mcp add github --env GITHUB_TOKEN=ghp_xxx
```

**Lister les serveurs :**

```
claude mcp list

# Output:
# ┌──────────────┬──────────┬──────────┐
# │ Name         │ Status   │ Tools    │
# ├──────────────┼──────────┼──────────┤
# │ postgres     │ Running  │ 5        │
# │ github       │ Running  │ 12       │
# │ chrome       │ Stopped  │ 0        │
# └──────────────┴──────────┴──────────┘
```

**Supprimer un serveur :**

```
claude mcp remove postgres
```

**Voir le status dans Claude Code :**

```
> /mcp

MCP Servers:
✓ postgres (5 tools)
✓ github (12 tools)
✗ chrome (not running)

Use /mcp auth to authenticate OAuth servers.
```

### 17.3 MCP Servers essentiels

```
claude mcp add postgres
```

**PostgreSQL : Requêtes en langage naturel** Configuration (.mcp.json) :

```
{
  "servers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://user:pass@localhost:5432/mydb"
      }
    }
  }
}
```

**Usage :**

```
> Montre-moi les utilisateurs qui se sont inscrits cette semaine

Claude utilise postgres_query :
SELECT * FROM users
WHERE created_at >= NOW() - INTERVAL '7 days'
ORDER BY created_at DESC
```

```
claude mcp add github
```

**GitHub : PRs, issues, reviews** **Outils disponibles :** - `github_create_issue` : Créer une issue - `github_create_pr` : Créer une PR - `github_list_prs` : Lister les PRs - `github_review_pr` : Reviewer une PR

**Usage :**

```
> Crée une PR pour merger feature/auth dans main

Claude utilise github_create_pr avec le bon titre et description.
```

```
claude mcp add chrome-devtools
```

**Chrome DevTools : Automatisation navigateur** **Outils disponibles :** - `navigate` : Aller à une URL - `screenshot` : Capturer l'écran - `click` : Cliquer sur un élément - `fill` : Remplir un formulaire - `evaluate` : Exécuter du JavaScript

**Usage :**

```
> Va sur localhost:3000 et fais une capture d'écran de la page d'accueil

Claude navigue et capture automatiquement.
```

**Autres serveurs populaires**

| Serveur | Usage |
|---|---|
| `sqlite` | Bases SQLite locales |
| `slack` | Envoyer des messages |
| `jira` | Gérer les tickets |
| `notion` | Accéder aux pages Notion |
| `filesystem` | Accès fichiers étendu |

### 17.4 Créer son propre MCP Server

**Structure d'un serveur MCP (Go) :**

```go
package main

import (
    "github.com/anthropics/mcp-go"
)

func main() {
    server := mcp.NewServer("my-server")

    // Enregistrer un outil
    server.RegisterTool(mcp.Tool{
        Name:        "my_tool",
        Description: "Description de mon outil",
        InputSchema: map[string]interface{}{
            "type": "object",
            "properties": map[string]interface{}{
                "query": map[string]interface{}{
                    "type":        "string",
                    "description": "La requête",
```

```
                },
            },
            "required": []string{"query"},
        },
        Handler: func(input map[string]interface{}) (interface{}, error) {
            query := input["query"].(string)
            // Traiter la requête
            return map[string]interface{}{
                "result": "Résultat pour: " + query,
            }, nil
        },
    })

    // Démarrer le serveur
    server.Run()
}
```

**Fichier manifest (mcp.json) :**

```
{
  "name": "my-server",
  "version": "1.0.0",
  "description": "Mon serveur MCP personnalisé",
  "transport": "stdio",
  "tools": [
    {
      "name": "my_tool",
      "description": "Description de mon outil"
    }
  ]
}
```

**Debugging :**

```
# Tester le serveur manuellement
echo '{"method":"tools/list"}' | ./my-mcp-server

# Logs
claude mcp logs my-server
```

**Exercice :**

1. Installez le serveur PostgreSQL MCP
2. Connectez-le à une base de données de test
3. Posez des questions en langage naturel sur les données
4. Testez le serveur GitHub si vous avez un token

---

## Module 18 : Skills et Subagents personnalisés (1h)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de créer des skills personnalisées avec frontmatter YAML, d'utiliser les variables ($ARGUMENTS, $PROJECT_DIR), de créer des subagents personnalisés, et de packager des plugins combinant skills, hooks et MCP servers.

### 18.1 Créer une Skill

**Emplacement des skills :**

```
.claude/
└── skills/
    ├── code-review.md
    ├── standup.md
    └── deploy.md
```

**Structure d'une skill :**

```
---
name: code-review
description: Effectue une revue de code approfondie
triggers:
  - /code-review
  - /review
---

# Instructions pour la revue de code

Tu es un reviewer de code expérimenté. Analyse le code fourni selon ces critères :

## Critères d'analyse

1. **Lisibilité**
   - Nommage des variables et fonctions
   - Structure et organisation
   - Commentaires pertinents

2. **Maintenabilité**
   - Principe DRY
   - Single Responsibility
   - Complexité cyclomatique

3. **Performance**
   - Algorithmes appropriés
   - Gestion mémoire
   - Requêtes optimisées

4. **Sécurité**
   - Validation des inputs
   - Gestion des erreurs
   - Pas de secrets hardcodés

## Format de sortie

Pour chaque fichier analysé, fournis :
- Score global /10
- Points positifs
- Points à améliorer (avec suggestions de code)
- Priorité des corrections (critique/important/mineur)

## Fichiers à analyser
```

```
$ARGUMENTS
```

**Variables disponibles :**

| Variable | Description |
|---|---|
| `$ARGUMENTS` | Ce qui suit la commande |
| `$PROJECT_DIR` | Dossier du projet |
| `$CURRENT_FILE` | Fichier actuel (si dans IDE) |

### 18.2 Skills avancées

**Frontmatter complet :**

```yaml
---
name: deploy
description: Déploie l'application sur Azure
triggers:
  - /deploy
  - /ship
model: sonnet           # Modèle à utiliser
timeout: 300            # Timeout en secondes
once: true              # Exécuter une seule fois par session
tools:
  allow:
    - Bash(az *)
    - Bash(docker *)
  deny:
    - Edit(*)           # Pas de modification de fichiers
---
```

**Skill multi-étapes :**

```yaml
---
name: release
description: Prépare et publie une release
triggers:
  - /release
---

# Processus de release

Exécute ces étapes dans l'ordre :

## Étape 1 : Vérification
- [ ] Tous les tests passent
- [ ] Pas de changements non commités
- [ ] Sur la branche main

## Étape 2 : Changelog
- Génère le changelog depuis la dernière release
- Demande confirmation à l'utilisateur

## Étape 3 : Version bump
- Incrémente la version selon semver
- Met à jour package.json / go.mod
```

```
## Étape 4 : Tag et push
- Crée le tag git
- Push avec les tags

## Étape 5 : Build et publish
- Build les artefacts
- Publie sur le registry

Demande confirmation avant chaque étape critique.
```

**Injection de contexte dynamique :**

```
---
name: standup
description: Génère un rapport de standup
triggers:
  - /standup
---

# Rapport de standup

Génère un rapport basé sur :

## Changements récents
```

$(gitlog --oneline --since = "yesterday" --author =$ "(git config user.email)“)

```
## Fichiers modifiés
```

$(git diff –stat HEAD~5)

```
## Format
- Ce que j'ai fait hier
- Ce que je fais aujourd'hui
- Blockers éventuels
```

### 18.3 Créer un Subagent personnalisé

**Emplacement des subagents :**

```
.claude/
└── agents/
    ├── security-auditor.md
    └── migration-helper.md
```

**Structure d’un subagent :**

```
---
name: security-auditor
description: Agent spécialisé en audit de sécurité
model: opus              # Utilise Opus pour le raisonnement
tools:
  - Read
  - Grep
  - Glob
  - WebSearch
```

```
---

# Security Auditor Agent

Tu es un expert en sécurité applicative. Ton rôle est d'auditer le code pour identifier les vul

## Ton processus

1. **Scan initial**
   - Cherche les patterns dangereux (eval, exec, SQL concat)
   - Identifie les inputs non validés
   - Repère les secrets potentiels

2. **Analyse approfondie**
   - Trace le flux des données utilisateur
   - Vérifie les autorisations
   - Analyse la gestion des sessions

3. **Rapport**
   - Liste les vulnérabilités trouvées
   - Classe par sévérité (critique/haute/moyenne/basse)
   - Propose des corrections

## Vulnérabilités à chercher

- Injection SQL
- XSS (Cross-Site Scripting)
- CSRF (Cross-Site Request Forgery)
- Injection de commandes
- Path traversal
- Désérialisation non sécurisée
- Secrets hardcodés

## Output

Retourne un rapport structuré avec :
- Résumé exécutif
- Liste détaillée des vulnérabilités
- Recommandations priorisées
```

**Appeler un subagent personnalisé :**

```
> Utilise l'agent security-auditor pour analyser src/api/

Claude lance le subagent et retourne le rapport.
```

### 18.4 Partager en équipe

**Structure de plugin partageable :**

```
my-team-plugin/
├── plugin.json          # Manifest du plugin
├── skills/
│   ├── code-review.md
│   └── standup.md
```

```
├── agents/
│   └── security-auditor.md
├── hooks/
│   └── pre-commit.sh
└── README.md
```

**Fichier plugin.json :**

```
{
  "name": "my-team-plugin",
  "version": "1.0.0",
  "description": "Plugin pour l'équipe X",
  "author": "Team X",
  "skills": [
    "skills/code-review.md",
    "skills/standup.md"
  ],
  "agents": [
    "agents/security-auditor.md"
  ],
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "command": ["./hooks/pre-commit.sh"]
      }
    ]
  }
}
```

**Installation d'un plugin équipe :**

```
# Depuis un repo git
claude plugins install git@github.com:team/our-plugin.git

# Depuis un dossier local
claude plugins install ./path/to/plugin
```

**Exercice :**

1. Créez une skill `/standup` pour votre workflow
2. Créez une skill `/code-review` avec vos critères
3. Testez les skills dans une session Claude Code
4. Bonus : créez un subagent spécialisé pour votre domaine

---

## Module 19 : Hooks - Automatisation événementielle (1h15)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'expliquer le système de hooks de Claude Code, de configurer des hooks pour différents événements (PreToolUse, PostToolUse, SessionStart), et d'implémenter des automatisations comme le formatage automatique ou les notifications.

### 19.1 Comprendre les Hooks

Les Hooks permettent d'exécuter des actions automatiquement en réponse à des événements Claude Code.

**Analogie :** - Git hooks (pre-commit, post-merge) - Webhooks (événements → actions) - Event listeners (on click, on change)

**Types de hooks :**

| Type | Description | Exemple |
|---|---|---|
| command | Script shell | Lancer un linter |
| prompt | Évaluation LLM | Vérifier avec un autre prompt |
| agent | Sous-agent | Analyse automatique |

### 19.2 Événements disponibles

**Cycle de vie de session :**

| Événement | Quand | Usage typique |
|---|---|---|
| `SessionStart` | Début de session | Charger du contexte |
| `SessionEnd` | Fin de session | Sauvegarder des notes |
| `Stop` | Fin de réponse | Notification |

**Interaction utilisateur :**

| Événement | Quand | Usage typique |
|---|---|---|
| `UserPromptSubmit` | Avant traitement du prompt | Validation, enrichissement |
| `PermissionRequest` | Demande de permission | Logging, auto-décision |

**Outils :**

| Événement | Quand | Usage typique |
|---|---|---|
| `PreToolUse` | Avant un outil | Validation, blocage |
| `PostToolUse` | Après un outil | Formatage, notification |

**Subagents :**

| Événement | Quand | Usage typique |
|---|---|---|
| `SubagentStart` | Lancement subagent | Logging |
| `SubagentStop` | Fin subagent | Analyse résultat |

### 19.3 Créer des Hooks

**Configuration dans settings.json :**

```
{
  "hooks": {
    "PostToolUse": [
      {
```

```
        "matcher": "Edit",
        "command": ["npm", "run", "format", "--", "$FILE"]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash(git push *)",
        "command": ["./scripts/pre-push-check.sh"]
      }
    ]
  }
}
```

**Hook command (script shell) :**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit(*.go)",
        "command": ["gofmt", "-w", "$FILE"],
        "timeout": 5000
      }
    ]
  }
}
```

Le script reçoit des variables d'environnement : - `$FILE` : Fichier concerné - `$TOOL` : Outil utilisé - `$CLAUDE_PROJECT_DIR` : Dossier projet

**Hook prompt (évaluation LLM) :**

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write(migrations/*)",
        "prompt": "Vérifie que cette migration est safe et réversible. Réponds OK ou explique
        "model": "haiku"
      }
    ]
  }
}
```

**Hook agent (sous-agent de vérification) :**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit(src/security/*)",
        "agent": "security-auditor",
        "timeout": 30000
      }
    ]
  }
```

```
}
```

### 19.4 Pièges courants avec les Hooks (Gotchas)

**Timeouts :**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "command": ["slow-script.sh"],
        "timeout": 30000  // 30 secondes, défaut = 5000ms
      }
    ]
  }
}
```

**Note :** Si le hook timeout, Claude continue sans bloquer.

**Exit codes :**

| Code | Signification |
|---|---|
| 0 | Succès, continuer |
| 2 | Bloquer l'action |
| Autre | Warning, continuer |

```
#!/bin/bash
# pre-push-check.sh

if grep -r "TODO" src/; then
  echo "TODOs found, blocking push"
  exit 2  # Bloque l'action
fi

exit 0  # OK, continue
```

**JSON parsing :**

Si votre hook retourne du JSON, il doit être valide :

```
#!/bin/bash
# Bon
echo '{"status": "ok"}'

# Mauvais - cassera le parsing
echo "Status: ok"
echo '{"status": ok}'  # Manque les guillemets
```

**Chemins relatifs :**

Utilisez toujours les variables d'environnement :

```
#!/bin/bash
# Mauvais
cd ./src && run-lint
```

```
# Bon
cd "$CLAUDE_PROJECT_DIR/src" && run-lint
```

**Hooks async :**

Les hooks async ne peuvent pas bloquer :

```
{
  "hooks": {
    "Stop": [
      {
        "command": ["./notify-slack.sh"],
        "async": true  // N'attend pas la fin
      }
    ]
  }
}
```

**Ordre d'exécution :**

Les hooks s'exécutent dans l'ordre de définition :

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit", "command": ["lint"] },    // 1er
      { "matcher": "Edit", "command": ["format"] },  // 2ème
      { "matcher": "Edit", "command": ["test"] }     // 3ème
    ]
  }
}
```

**Debug :**

```
# Voir l'exécution des hooks
claude --debug

# Logs détaillés
CLAUDE_DEBUG=1 claude
```

### 19.5 Exemples de hooks utiles

**Auto-format après édition :**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit(*.go)",
        "command": ["gofmt", "-w", "$FILE"]
      },
      {
        "matcher": "Edit(*.{ts,tsx,js,jsx})",
        "command": ["npx", "prettier", "--write", "$FILE"]
      }
    ]
```

```
  }
}
```

**Lint check avant commit :**

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash(git commit *)",
        "command": ["npm", "run", "lint"],
        "timeout": 30000
      }
    ]
  }
}
```

**Secret scanner :**

```
#!/bin/bash
# scripts/scan-secrets.sh

# Patterns de secrets
PATTERNS="(password|secret|api_key|token)\\s*=\\s*['\"][^'\"]+['\"]"

if grep -rE "$PATTERNS" "$1" 2>/dev/null; then
  echo "ERREUR: Secrets potentiels détectés!"
  exit 2
fi

exit 0
```

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash(git add *)",
        "command": ["./scripts/scan-secrets.sh", "$CLAUDE_PROJECT_DIR"]
      }
    ]
  }
}
```

**Changelog automatique :**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash(git commit *)",
        "prompt": "Analyse ce commit et ajoute une entrée appropriée dans CHANGELOG.md si c'est
      }
    ]
  }
}
```

**Notification Slack :**

```bash
#!/bin/bash
# scripts/notify-slack.sh

curl -X POST -H 'Content-type: application/json' \
  --data "{\"text\":\"Claude a modifié: $FILE\"}" \
  "$SLACK_WEBHOOK_URL"
```

**Context injection :**

```json
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "command": ["./scripts/inject-context.sh"],
        "output": "prepend"
      }
    ]
  }
}
```

```bash
#!/bin/bash
# scripts/inject-context.sh
echo "Current branch: $(git branch --show-current)"
echo "Last commit: $(git log -1 --oneline)"
echo "Date: $(date)"
```

### 19.6 Claude Code SDK

Pour des intégrations plus complexes, utilisez le SDK.

**SDK TypeScript :**

```typescript
import { ClaudeCode } from '@anthropic-ai/claude-code-sdk';

const claude = new ClaudeCode({
  apiKey: process.env.ANTHROPIC_API_KEY,
});

// Session interactive
const session = await claude.createSession({
  workingDirectory: './my-project',
});

// Envoyer un message
const response = await session.send('Analyse ce projet');

// Écouter les événements
session.on('toolUse', (tool, params) => {
  console.log(`Tool used: ${tool}`);
});

// Fermer
await session.close();
```

**SDK Python :**

```
from claude_code import ClaudeCode

claude = ClaudeCode(api_key=os.environ['ANTHROPIC_API_KEY'])

with claude.session(working_dir='./my-project') as session:
    response = session.send('Analyse ce projet')
    print(response.content)

    # Mode streaming
    for chunk in session.stream('Explique le fichier main.go'):
        print(chunk, end='')
```

**Cas d'usage SDK :** - Bots Discord/Slack - Pipelines CI/CD avancés - Outils de développement custom - Intégration dans des applications

**Exercice :**

1. Créez un hook de formatage automatique pour votre langage
2. Créez un hook de validation avant git push
3. Testez avec `claude --debug` pour voir l'exécution
4. Bonus : créez un hook de notification (email, Slack, etc.)

---

## Module 20 : IDE Integrations (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'installer et utiliser l'extension VS Code et le plugin JetBrains, d'activer Chrome DevTools pour les tests UI, et de choisir la bonne interface (IDE vs terminal) selon le contexte.

### 20.1 Extension VS Code

**Installation :** VS Code → Extensions → "Claude Code" → Installer

**Fonctionnalités clés :** - **Prompt Box** : Zone de saisie dans le panneau latéral - **File References** : @mentions depuis l'éditeur - **Selection Context** : Envoyer la sélection à Claude - **Multi-conversation** : Plusieurs chats en parallèle - **Basculer terminal** : `Claude: Open in Terminal`

### 20.2 Plugin JetBrains

**IDEs supportés :** IntelliJ, PyCharm, GoLand, WebStorm, PhpStorm, Rider

**Installation :** Settings → Plugins → Marketplace → "Claude Code"

**Support Remote :** SSH, Docker containers, WSL

### 20.3 Chrome Integration

```
claude --chrome      # Activer Chrome DevTools
claude --no-chrome   # Désactiver
```

**Cas d'usage :** Tests E2E, screenshots, scraping, formulaires

### 20.4 Bonnes pratiques IDE

| Situation | Recommandation |
|---|---|
| Modification rapide | IDE |
| Exploration projet | Terminal (subagent) |
| Refactoring multi-fichiers | Terminal |

---

## Module 21 : Mode Headless et Automatisation (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'exécuter Claude Code en mode non-interactif avec –p, de configurer les options avancées (max-turns, budget, output-format), et d'intégrer Claude Code dans des scripts shell.

### 21.1 Print Mode (–p)

```
claude -p "Liste les TODO"              # Exécuter et quitter
claude -p "query" --output-format json  # Sortie JSON
```

### 21.2 Options avancées

```
claude -p "Fix bug" --max-turns 5 --max-budget-usd 1.00 --no-session-persistence
```

### 21.3 Script exemple

```
#!/bin/bash
commits=$(git log --oneline ${last_tag}..HEAD)
claude -p "Changelog pour: $commits" > CHANGELOG.md
```

### 21.4 SDK

**TypeScript/Python :** Sessions programmatiques, streaming, événements

---

## Module 22 : CI/CD - GitHub Actions et GitLab (1h)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de configurer Claude Code dans un pipeline GitHub Actions ou GitLab CI, d'automatiser les code reviews sur les PR/MR, et de générer des changelogs automatiques.

### 22.1 GitHub Actions

```
- uses: anthropics/claude-code-action@v1
  with:
    anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
    prompt: "Review this PR"
```

### 22.2 GitLab CI

```
script:
  - npm install -g @anthropic-ai/claude-code
  - claude -p "Review MR"
```

---

## Module 23 : Patterns agentiques avancés (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'utiliser le Plan Mode pour des tâches complexes, d'orchestrer plusieurs subagents (parallèle vs séquentiel), et d'activer Extended Thinking pour les problèmes algorithmiques difficiles.

### 23.1 Plan Mode

```
claude --permission-mode plan
```

Workflow : Exploration → Plan → Validation → Implémentation

### 23.2 Orchestration subagents

- Parallèle : tâches indépendantes
- Séquentiel : tâches dépendantes

### 23.3 Extended Thinking

`Alt+T` ou `export MAX_THINKING_TOKENS=10000`

---

## Module 24 : Optimisation et Productivité (45 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable d'optimiser l'utilisation des tokens, de réduire les coûts avec des prompts efficaces et le bon choix de modèle, et d'appliquer les bonnes pratiques pour des sessions productives.

### Gestion contexte

```
/context  → Visualiser usage
/compact  → Compresser conversation
```

### Réduction tokens

- Prompts spécifiques
- Skills pour tâches répétitives
- Déléguer aux subagents
- Bon choix de modèle

---

## Module 25 : Configuration Terminal et UX (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de personnaliser l'interface terminal (thème, mode vim, style), de configurer des keybindings personnalisés, et d'adapter Claude Code à ses préférences d'utilisation.

```
/theme monokai    → Changer thème
/vim enable       → Mode vim
/style concise    → Style bref
```

Keybindings : `~/.claude/keybindings.json`

---

## Module 26 : Déploiement Enterprise (30 min)

**Objectif pédagogique** : À l'issue de ce module, l'apprenant sera capable de configurer Claude Code en contexte enterprise avec managed settings, d'utiliser les providers cloud (AWS Bedrock, GCP Vertex), et de gérer les contraintes réseau (proxy, mTLS, LLM gateways).

### Managed Settings

Fichier `managed-settings.json` contrôlé par IT

### Providers

```
export ANTHROPIC_PROVIDER=bedrock   # AWS
export ANTHROPIC_PROVIDER=vertex    # GCP
```

### Réseau

Proxy, mTLS, LLM Gateways

---

## Projet intermédiaire : GitBar - Outil desktop rapide (2h)

**Objectif pédagogique** : Démontrer comment Claude Code permet de créer rapidement un outil desktop personnel en Go avec Wails, de l'idée au binaire fonctionnel.

### Présentation

**GitBar** est une application de barre de menu pour surveiller vos repositories GitHub/GitLab : PRs ouvertes, statut des pipelines CI, notifications sur les événements importants.

### Stack technique

```
Backend : Go 1.22+
Frontend : HTML/CSS/JS (ou Svelte/React/Vue)
Framework : Wails v2
API : GitHub REST API / GitLab API
```

**Fonctionnalités cibles**

1. **Icône barre de menu** avec indicateur de statut (vert/orange/rouge)
2. **Liste des PRs/MRs** à reviewer
3. **Statut des pipelines CI** (GitHub Actions / GitLab CI)
4. **Notifications natives** macOS/Windows/Linux
5. **Configuration** : token API, repos à surveiller

**Workflow avec Claude Code**

**Étape 1 : Initialisation projet Wails**

```
> Initialise un projet Wails nommé "gitbar" avec Go backend.
> Utilise le template vanilla (HTML/CSS/JS simple).
> Crée la structure de base.
```

**Étape 2 : Configuration API GitHub**

```
> Ajoute la configuration pour l'API GitHub :
> - Lecture du token depuis variable d'environnement ou fichier config
> - Structure pour stocker la liste des repos à surveiller
> - Client HTTP avec authentification Bearer
```

**Étape 3 : Récupération des PRs**

```
> Crée une fonction Go qui récupère les Pull Requests ouvertes
> pour les repos configurés. Retourne :
> - Titre, auteur, date de création
> - Statut des checks CI
> - Nombre de reviews approuvées/demandées
```

**Étape 4 : Interface utilisateur**

```
> Crée l'interface HTML/CSS pour afficher :
> - Liste des PRs groupées par repo
> - Badge de statut coloré (vert=ready, orange=pending, rouge=failing)
> - Lien cliquable vers la PR sur GitHub
> Style moderne et minimaliste, thème sombre.
```

**Étape 5 : Système de polling**

```
> Implémente un polling toutes les 60 secondes pour rafraîchir les données.
> Expose une méthode Wails pour déclencher un refresh manuel.
> Gère les erreurs réseau gracieusement.
```

**Étape 6 : Notifications natives**

```
> Ajoute les notifications desktop natives quand :
> - Une nouvelle PR est assignée pour review
> - Un pipeline échoue sur une de mes PRs
> Utilise la librairie beeep ou similaire pour Go.
```

**Étape 7 : Build et distribution**

```
> Configure le build Wails pour :
> - macOS : app bundle avec icône personnalisée
> - Windows : exe avec icône
> - Linux : binaire AppImage
> Crée un Makefile avec les commandes de build.
```

**Structure du projet**

```
gitbar/
├── main.go                 # Point d'entrée Wails
├── app.go                  # Logique backend exposée au frontend
├── internal/
│   ├── github/
│   │   ├── client.go       # Client API GitHub
│   │   └── types.go        # Structures de données
│   ├── config/
│   │   └── config.go       # Gestion configuration
│   └── notify/
│       └── notify.go       # Notifications natives
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── main.js
├── build/
│   └── appicon.png
├── wails.json
└── CLAUDE.md               # Instructions pour Claude Code
```

**CLAUDE.md pour GitBar**

```
# CLAUDE.md - GitBar

## Projet
Application desktop barre de menu pour surveiller GitHub PRs et CI.

## Stack
- Backend: Go 1.22, Wails v2
- Frontend: Vanilla JS, CSS moderne
- API: GitHub REST API v3

## Commandes
- `wails dev` : Mode développement avec hot-reload
- `wails build` : Build production
- `make build-all` : Build toutes plateformes

## Conventions Go
- Erreurs wrappées avec fmt.Errorf
- Context pour les appels API
- Structured logging avec slog

## Points d'attention
- Token GitHub jamais hardcodé
- Gestion du rate limiting API
- Timeout sur les appels HTTP (10s)
```

**Livrables**

- ☐ Projet Wails initialisé et fonctionnel

- ☐ Connexion API GitHub avec token
- ☐ Affichage des PRs ouvertes
- ☐ Indicateur de statut CI
- ☐ Notifications sur événements importants
- ☐ Build fonctionnel pour votre OS
- ☐ CLAUDE.md adapté au projet

#### Points d'apprentissage

Ce projet démontre : 1. **Prototypage rapide** : De l'idée à l'outil fonctionnel en ~2h 2. **Stack Go moderne** : Wails pour le desktop natif 3. **Intégration API** : GitHub/GitLab avec authentification 4. **CLAUDE.md efficace** : Configuration projet ciblée

---

### Projet final : BizCraft SaaS Complet

#### Phase 1 — MVP

- Auth Microsoft Entra External ID
- Dashboard E/M/T
- Process Builder (SOPs)

#### Phase 2 — Avancé

- Gestion rôles / organigramme
- KPIs avec Chart.js
- Mode Coach multi-tenant

#### Évaluation

| Critère | Poids |
|---|---|
| Qualité code | 20% |
| Features Claude Code | 25% |
| Configuration projet | 20% |
| CI/CD | 15% |
| Architecture/sécurité | 10% |
| Documentation | 10% |

---

## RESSOURCES

- Templates CLAUDE.md par stack
- 20+ skills prêtes à l'emploi
- 10+ hooks de validation
- Config MCP servers
- Starter kit BizCraft
- Cheatsheet PDF
- Accès Discord

**Liens :** - Documentation officielle - Best Practices - GitHub Claude Code

---

# ANNEXE : RÉFÉRENCE RAPIDE

## CLI Flags

| Flag | Description |
|---|---|
| `-c` | Continuer dernière session |
| `-r <id>` | Reprendre session |
| `-p "..."` | Mode headless |
| `--model` | Choisir modèle |

## Slash Commands

| Commande | Description |
|---|---|
| `/help` | Aide |
| `/clear` | Effacer |
| `/commit` | Git commit |
| `/compact` | Compresser |
| `/rewind` | Revenir arrière |

## Raccourcis

| Raccourci | Action |
|---|---|
| `Ctrl+C` | Annuler |
| `Ctrl+D` | Quitter |
| `Esc Esc` | Rewind |
| `Alt+P` | Changer modèle |
| `Alt+T` | Extended Thinking |

---

*Dernière mise à jour : 2026-02-04*